CLASSIFICAÇÃO E PREDIÇÃO

Sumário

NOSSA HISTÓRIA 3

CLASSIFICAÇÃO E PREDIÇÃO 4

Definição de Classe 7

FERRAMENTAS DE CLASSIFICAÇÃO E PREDIÇÃO 8

Redes Neurais 9

Redes Neurais: Radial Basis Function (RBF) 15

Árvores de decisão 17

Classificadores Bayesianos 22

O Classificador Naive Bayes 24

SVM (Support Vector Machines) 27

Classificação por Regras de Associação (Classification by Association Rule) 27

Aprendizado Tardio (Lazy Learners) 27

Algoritmo Genético (Genetic Algorithm) 28

Conjuntos Aproximados (Rought Set) 28

Conjuntos Nebulosos (Fuzzy Set) 29

Regressão Linear 29

Regressão Não-Linear 30

ESTUDO DE CASO I 31

Classificação e Predição de Preços de Imóveis a Partir de Dados Estruturados e Não Estruturados 31

Introdução 31

Fundamentos 32

Pré-processamento de dados 32

Dados estruturados 33

Dados não estruturados 34

Extração de texto a partir de HTML 34

Técnicas para tratamento de texto 35

Aprendizado supervisionado 35

Algoritmos de aprendizado supervisionado 37
Random Forest 37
Support Vector Machine – Regression 37
XGBoost 38
Métricas de avaliação de um modelo de regressão 38
IMPLEMENTAÇÃO DO REGRESSOR 40
Dados Utilizados e Pré-processamento 41
ANÁLISE DOS RESULTADOS 47
Configuração do ambiente 47
Resultados 48
Dados Estruturados 48
Combinação dos Dados 50
Conclusão 52
ESTUDO DE CASO II 53
Predição de Alunos com Risco de Evasão: estudo de caso usando mineração de dados 53
Introdução 54
Resultados Obtidos 56
REFERÊNCIAS 59

NOSSA HISTÓRIA

A nossa história inicia-se com a ideia visionária e da realização do sonho de um grupo de empresários na busca de atender à crescente demanda de cursos de Graduação e Pós-Graduação. E assim foi criado o Instituto, como uma entidade capaz de oferecer serviços educacionais em nível superior.

O Instituto tem como objetivo formar cidadão nas diferentes áreas de conhecimento, aptos para a inserção em diversos setores profissionais e para a participação no desenvolvimento da sociedade brasileira, e assim, colaborar na sua formação continuada. Também promover a divulgação de conhecimentos científicos, técnicos e culturais, que constituem patrimônio da humanidade, transmitindo e propagando os saberes através do ensino, utilizando-se de publicações e/ou outras normas de comunicação.

Tem como missão oferecer qualidade de ensino, conhecimento e cultura, de forma confiável e eficiente, para que o aluno tenha oportunidade de construir uma base profissional e ética, primando sempre pela inovação tecnológica, excelência no atendimento e valor do serviço oferecido. E dessa forma, conquistar o espaço de uma das instituições modelo no país na oferta de cursos de qualidade.

CLASSIFICAÇÃO E PREDIÇÃO

A classificação é o processo de encontrar um conjunto de modelos que descrevem e distinguem classes de dados ou conceitos. Esses modelos são usados para predição de objetos cujas classes são desconhecidas, baseada na análise de um conjunto de dados de treinamento (objetos cujas classes são conhecidas).

Segundo Soares (2005), a tarefa de classificação tem como objetivo encontrar algum tipo de relacionamento entre os atributos preditivos e o atributo objetivo, de modo a obter um conhecimento que possa ser utilizado para prever a classe de um determinado registro que ainda não possui classe definida.

O modelo gerado pode ser representado sob a forma de regras de classificação (se-então), árvores de decisão, fórmulas matemáticas ou redes neurais. As tarefas de classificação e predição podem requerer análise de relevância para identificar atributos que não contribuem para esses processos, os quais poderão, portanto, ser excluídos.

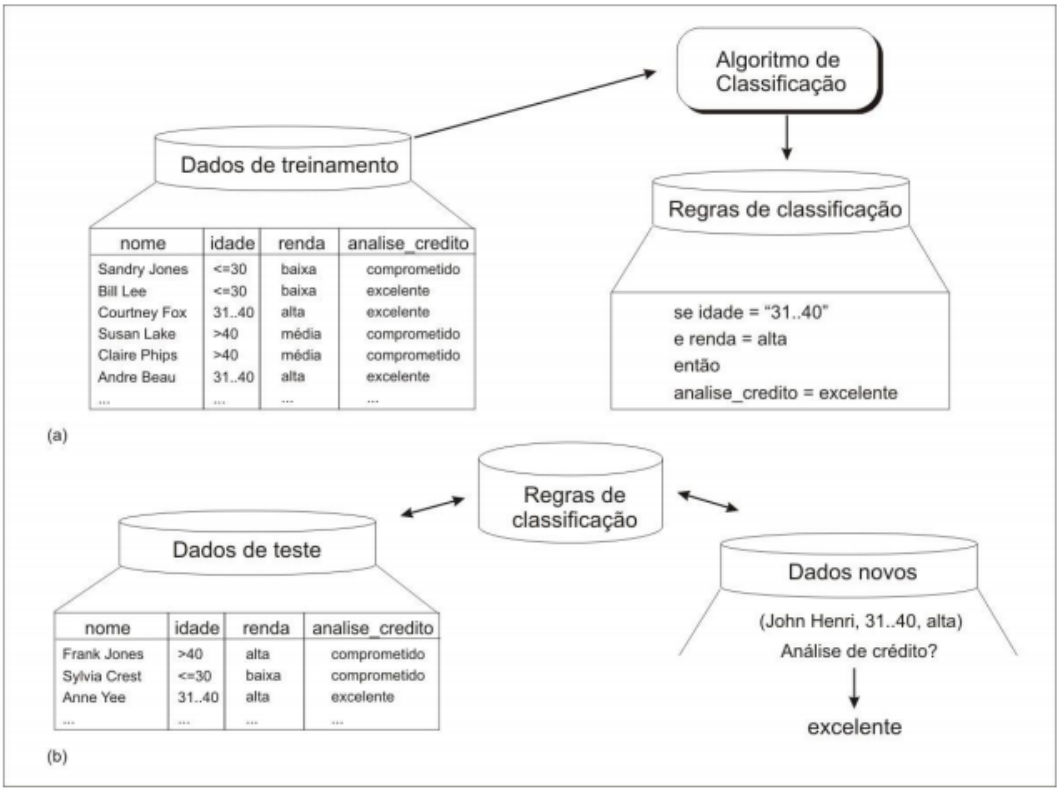
Conforme Ferreira (2005), um problema de classificação de padrões se resume a determinar, da forma mais correta possível, a classe de saída à qual o padrão de entrada (registro) pertence. O desenvolvimento de um modelo de classificação caracteriza-se pela definição do grupo de classes, das varáveis relevantes e de um conjunto de treinamento consistindo em exemplos (padrões) pré-classificados.

Grande parte dos problemas de classificação de padrões de interesse real tem duas fases: uma etapa in sample ou de treinamento (aprendizado), executada a partir do banco de dados existente, e uma etapa out of sample ou de generalização, onde são apresentados dados que não foram utilizados no treinamento.

O que se deseja, fundamentalmente, é extrair do banco de dados as características que definem o mapeamento entre entrada e saída, para posteriormente utilizá-las na tomada de decisões em dados novos, ainda não avaliados pelo sistema (FERREIRA, 2005).

Han et al. (2001) descreve o primeiro passo em classificação como a construção de um conjunto pré-determinado de classes e conceitos. O modelo é construído através da análise de tuplas de um banco de dados descritas como atributos. Cada tupla é assumida como pertencente a determinada classe, como determinado por um dos atributos que formam a base – este atributo é denominado atributo de rótulo de classe (ou class label attribute). Neste contexto, as tuplas ou registros também são denominadas amostras, exemplos ou objetos.

Estas amostras selecionadas para o modelo formam o data mart (ou data set) de treinamento. A Figura 4.1 (a) ilustra este processo. Os registros ou amostras individuais que formam o conjunto de dados de treinamento são denominados amostras de treino e são selecionadas aleatoriamente da população total (recomenda-se a utilização de técnicas de redução de dados e amostragem para a seleção destes objetos).



Como o rótulo (ou descrição) de cada atributo é conhecido e fornecido ao modelo, esta etapa de classificação também é denominada aprendizado supervisionado (a aprendizagem do modelo é dita "supervisionada" porque se conhece a classe a que pertence cada um dos objetos do conjunto de

treinamento). Tomando como exemplo o conjunto de dados utilizado na formação do modelo proposto neste trabalho, para cada registro (ou objeto) do conjunto de treinamento, formado por diversos atributos referentes a cada cliente (como dados cadastrais, dados de consumo e de faturamento), são conhecidos os registros ou amostras pertencentes à classe alvo da predição (ou seja, a classe churner – clientes que efetuaram o cancelamento do serviço junto à operadora).

O aprendizado supervisionado contrasta com o aprendizado não supervisionado (ou clustering), onde o rótulo dos atributos em cada amostra de treino não é conhecido, e o número ou conjunto de classes a serem aprendidas pode não ser conhecidas previamente. Este trabalho utiliza a metodologia de classificação e predição para a identificação da classe objetiva, ou seja, clientes potenciais a cancelarem o serviço (também denominados churners). Assim, o modelo definido baseia-se na utilização de uma metodologia de classificação / predição, com aprendizado supervisionado.

Em um segundo passo, o modelo é utilizado para classificação (Figura 4.1 – b). Neste momento, estima-se a precisão do modelo (ou classificador). Os principais critérios para avaliar a precisão do modelo são (HAN et al., 2001):

• precisão da predição: refere-se a habilidade do modelo em corretamente predizer a classificação de objetos novos ou ainda não conhecidos;

• velocidade: refere-se aos custos computacionais em geral envolvidos na utilização do modelo;

• robustez: habilidade do modelo em efetuar predições corretas mesmo em um modelo contendo dados com ruído ou valores ausentes;

• escalabilidade: habilidade para construir um modelo eficiente em função da grande quantidade de dados;

• interpretabilidade: refere-se ao nível de informação útil e conhecimento descoberto e provido pelo modelo.

A precisão de um modelo em um determinado conjunto de dados de treinamento é a porcentagem de amostras de teste que são corretamente

classificadas pelo modelo. Para cada amostra do conjunto de teste, o valor conhecido da classe (real) é comparado com o valor previsto pelo algoritmo. Se a precisão do modelo é considerada aceitável, o modelo pode ser utilizado para classificar objetos futuros (onde não se conhece o valor da classe). Considerando este trabalho, esta etapa seria a utilização do modelo para a predição de clientes churners no ambiente organizacional (ou a aplicação comercial da técnica), buscando a predição da classe para um conjunto de dados não utilizado na fase de aprendizado.

Definição de Classe

Uma importante questão é o entendimento do conceito de classes nos estudos de classificações, considerando a natureza e o caminho que elas são definidas. Existem três casos bem definidos de classes, descritos em (MICHIE et al., 1994):

▪ classes como rótulos de diferentes populações: neste tipo de classe não leva em consideração associações de várias populações. Por exemplo, cachorro e gato formam duas diferentes classes ou populações, e como é conhecido, com certeza, ou o animal é um cachorro ou um gato (ou nenhum). Associação de uma classe ou população é determinada por uma autoridade independente, a distribuição para uma classe é independentemente determinada de quaisquer atributos ou variáveis;

▪ classes resultantes de um problema de predição: esta classe é uma conseqüência do resultado de uma predição através do conhecimento dos atributos. Em termos estatísticos, a classe é uma variável randômica. Um exemplo típico é em aplicações de predições de taxas de juros. Freqüentemente a questão é: as taxas de juros subirão (classe = 1) ou não (classe = 0)?;

▪ classes determinadas pelos valores espaciais dos seus atributos: pode-se dizer que a classe é uma função dos atributos. Então um item manufaturado pode ser classificado como falho se alguns atributos estão fora dos limites pré-determinados, e não falhos de outra forma. Existem regras que classificam os valores dos atributos. O problema é criar uma regra que imite a realidade o mais próximo possível. Muitos sistemas de crédito são desse tipo de classe.

No modelo proposto neste trabalho, a classe é definida claramente como resultante de um problema de predição, podendo ser descrita como um atributo pertencente à classe Churner (C) ou Não-Churner (NC), em função do resultado da aplicação de um algoritmo de classificação.

FERRAMENTAS DE CLASSIFICAÇÃO E PREDIÇÃO

Em um projeto de mineração de dados, com foco definido em Classificação e Predição, a utilização de diferentes abordagens e algoritmos é fundamental para um processo de avaliação. Como descrito por Han et al. (2001), são avaliados não apenas a precisão do modelo, mas também a velocidade, a robustez, a escalabilidade e a interpretabilidade.

Dependendo do modelo e da estrutura de predição em um cenário de aprendizado supervisionado, alguns algoritmos e ferramentas podem ser mais adequados ao problema em questão. Este capítulo apresenta um estudo exploratório sobre as principais técnicas e algoritmos de Classificação / Predição, em função do núcleo do modelo proposto. Serão avaliados três métodos, citados principalmente por Han et al. (2001), Fayyad et al. (1996) e Klösgen et al. (2002):

❑ Redes Neurais – particularmente o modelo RBF Network – Radial Basis Function Network;

❑ Decision Trees (Árvores de Decisão) – particularmente o algoritmo C4.5 (através do classificador J48, da ferramenta WEKA);

❑ Classificadores Bayesianos (principalmente o método Naive Bayes).

A escolha destes algoritmos como ferramentas de predição a serem utilizadas neste trabalho foi embasada nos seguintes aspectos:

▪ disponibilidade de uso – tais algoritmos são encontrados no pacote de classificadores da ferramenta WEKA (Waikato Environment for Knowledge Analysis), utilizada no estudo de caso deste trabalho;

▪ performance: em testes preliminares, os algoritmos explorados neste capítulo foram os que apresentaram melhor desempenho sobre o conjunto de dados do modelo;

▪ custo: avaliado principalmente sob a ótica dos recursos necessários para a execução dos algoritmos. Por exemplo, a utilização de Redes Neurais MLP´s (MultiLayer Perceptron), foi descartada em função do tempo de processamento e dos recursos de hardware exigidos para o modelo em questão.

Redes Neurais

Segundo Klösgen et al. (2002), Redes Neurais oferecem uma arquitetura computacional robusta e distribuída, composta de significativas funções de aprendizado e capacidade de representação de relacionamentos não-lineares e multi-variáveis.

Conforme Coutinho (2003), essa tecnologia é a que oferece o mais profundo poder de mineração, mas é, também, a mais difícil de se entender. As redes neurais tentam construir representações internas de modelos ou padrões achados nos dados, mas essas representações não são apresentadas para o usuário. Com elas, o processo de descoberta de padrões é tratado pelos programas de data mining dentro de um processo “caixa preta”.

Estruturalmente, uma rede neural consiste em um número de elementos interconectados (chamados neurônios) organizados em camadas que aprendem pela modificação da conexão que conectam as camadas (LEMOS, 2003).

As Redes Neurais Artificiais utilizam um conjunto de elementos de processamento (ou nós) análogos aos neurônios no cérebro humano. Estes elementos de processamento são interconectados em uma rede que pode identificar padrões nos, ou seja, a rede aprende através da experiência. Esta característica distingue redes neurais de tradicionais programas computacionais, que simplesmente seguem instruções em uma ordem seqüencial fixa (DIN, 1998).

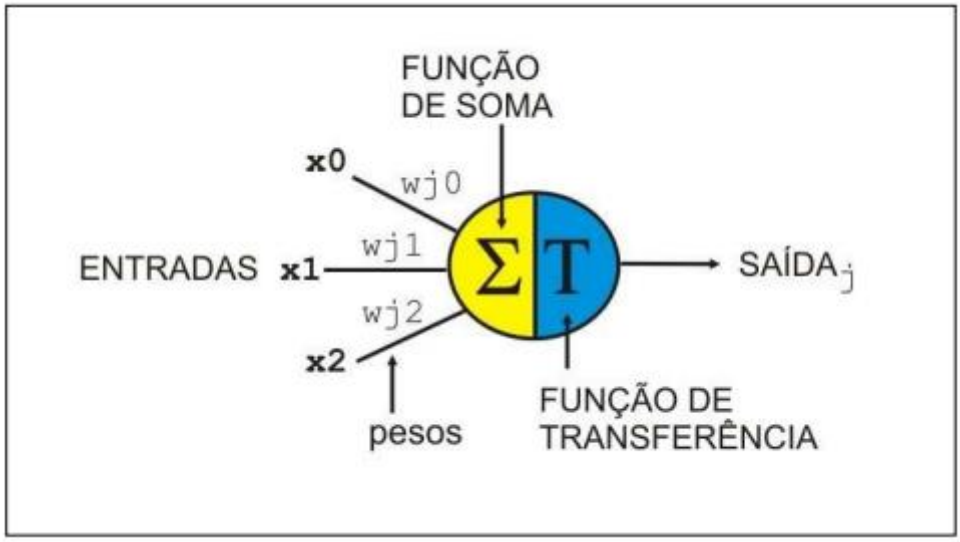
Embora seja verdadeiro que as redes neurais apresentem o mais avançado poder de mineração, muitos analistas de negócio não podem fazer uso

delas porque os resultados finais não podem ser explicados. Estruturalmente, uma rede neural consiste em um número de elementos interconectados (chamados neurônios), organizados em camadas que aprendem pela modificação da conexão firmemente conectando as camadas. Geralmente, constroem superfícies equacionais complexas através de interações repetidas, cada hora ajustando os parâmetros que definem a superfície. Depois de muitas repetições, uma superfície pode ser internamente definida e se aproxima muito dos pontos dentro do grupo de dados (CISTER, 2005).

As redes neurais artificiais consistem em um método para solucionar problemas da área de inteligência artificial, através da construção de um sistema que tenha circuitos que simulem o cérebro humano, inclusive seu comportamento, ou seja, aprendendo, errando e fazendo descobertas. São técnicas computacionais que apresentam um modelo inspirado na estrutura neural dos organismos inteligentes, que adquirem conhecimento através da experiência (LEMOS, 2003).

Assim como o sistema nervoso é composto de bilhões de células nervosas, a rede neural artificial também seria formada por unidades que nada mais são do que pequenos módulos (ou unidades de processamento ou nós) que simulam o funcionamento de um neurônio. Estes módulos devem funcionar de acordo com os elementos em que foram inspirados, recebendo e retransmitindo informações (LEMOS, 2003).

O neurônio artificial é uma estrutura lógico-matemática que procura simular a forma, o comportamento e as funções de um neurônio biológico. Assim sendo, os dendritos foram substituídos por entradas, cujas ligações com o corpo celular artificial são realizadas através de elementos chamados de peso (simulando as sinapses). Os estímulos captados pelas entradas são processados pela função de soma, e o limiar de disparo do neurônio biológico foi substituído pela função de transferência (TAFNER, 1998). A Figura 5.1 ilustra a estrutura de um neurônio artificial.



As funções básicas de cada neurônio são: (a) avaliar valores de entrada, (b) calcular o total para valores de entrada combinados, (c) comparar o total com um valor limiar, (d) determinar o que será a saída.

Segundo Kröse et al. (1996), o vetor x que representa um conjunto de "n" entradas, é multiplicado por um vetor de pesos, w, e o produto, p = x w, é aplicado aos canais de entrada do neurônio.

A soma de todas as entradas ponderadas é então processada por uma função de ativação, F(x), que vai produzir o sinal de saída a, do neurônio:

$$a = F\left(\sum_{i=0}^{n-1} x_i w_i + \theta\right)$$

O parâmetro θ é um valor threshold adicionado a soma ponderada, e em alguns casos é omitido, enquanto que em outros é considerado como o valor cujo peso correspondente ao valor de entrada é sempre igual a 1.

Segundo Kröse et al. (1996), o papel de θ, chamado de bias ou vício, é aumentar o número de graus de liberdade disponíveis no modelo, permitindo que a rede neural tenha maior capacidade de se ajustar ao conhecimento a ela fornecido.

A função de ativação é importante para o comportamento de uma rede neural porque é ela que define a saída do neurônio artificial e, portanto, o caminho pelo qual a informação é conduzida (KRÖSE et al., 1996).

É através de uma função de ativação que são calculadas as respostas geradas pelas unidades. Existem vários tipos de funções de ativação, sendo que as mais comuns são as descritas a seguir (KRÖSE et al., 1996).

▪ Função Passo, que produz uma saída binária, e embora seja similar aos neurônios reais, é inadequada para o algoritmo de aprendizagem;

▪ Função Linear, que elimina a descontinuidade em $x = \theta$;

▪ Função Sigmoidal, que adiciona alguma não-linearidade.

Enquanto a operação de cada neurônio é razoavelmente simples, procedimentos complexos podem ser criados pela conexão de um conjunto de neurônios. Tipicamente, as entradas dos neurônios são ligadas a uma camada intermediária (ou várias camadas intermediárias) e esta conectada à camada de saída.

Para construir um modelo neural, primeiramente, "adestra-se" a rede em um dataset de treinamento e, então, usa-se a rede já treinada para fazer predições. O dataset pode ser monitorado durante a fase de treinamento para checar seu progresso.

Cada neurônio, geralmente, tem um conjunto de pesos que determina como avaliase a combinação dos sinais de entrada. A entrada para um neurônio pode ser positiva ou negativa. O aprendizado se faz pela modificação dos pesos usados pelo neurônio em acordo com a classificação de erros que foi feita pela rede como um todo. As entradas são, geralmente, pesadas e normalizadas para produzir um procedimento suave.

Em uma rede neural artificial as entradas, simulando uma área de captação de estímulos, podem ser conectadas em muitos neurônios, resultando, assim, em uma série de saídas, onde cada neurônio representa uma saída. Essas conexões, em comparação com o sistema biológico, representam o contato dos dendritos com outros neurônios, formando assim as sinapses.

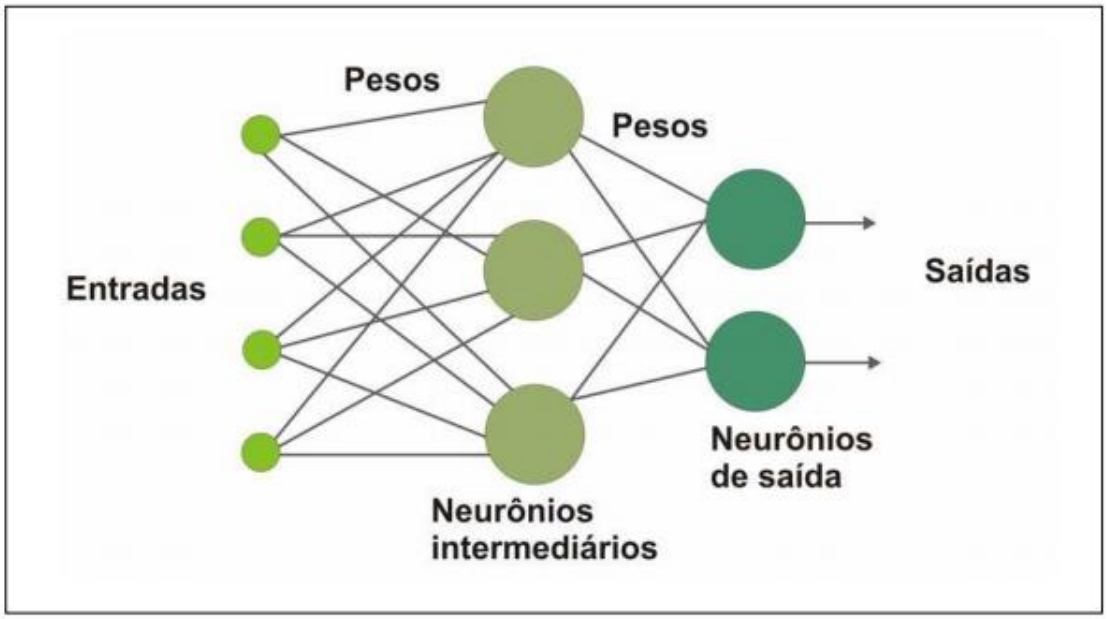
A função da conexão em si é tornar o sinal de saída de um neurônio em um sinal de entrada de outro, ou ainda, orientar o sinal de saída. As diferentes possibilidades de conexões entre as camadas de neurônios podem ter, em geral, n números de estruturas diferentes (TAFNER, 1998).

Usualmente, trabalha-se com três camadas, que são classificadas em:

▪ Camada de Entrada: onde os padrões são apresentados à rede;

▪ Camadas Intermediárias ou Ocultas: onde é feita a maior parte do processamento, através das conexões ponderadas; podem ser consideradas como extratoras de características;

▪ Camada de Saída: onde o resultado final é concluído e apresentado.



Existe uma significativa quantidade de variantes de uma rede neural, e podem ser modeladas conforme a aplicação. O que faz as redes neurais diferirem entre si são os tipos de conexões e formas de treinamento. Basicamente, os itens que compõem uma rede neural e, portanto, sujeitos a modificações, são os seguintes:

• forma de conexões entre camadas; • número de camadas intermediárias;

• quantidade de neurônios em cada camada;

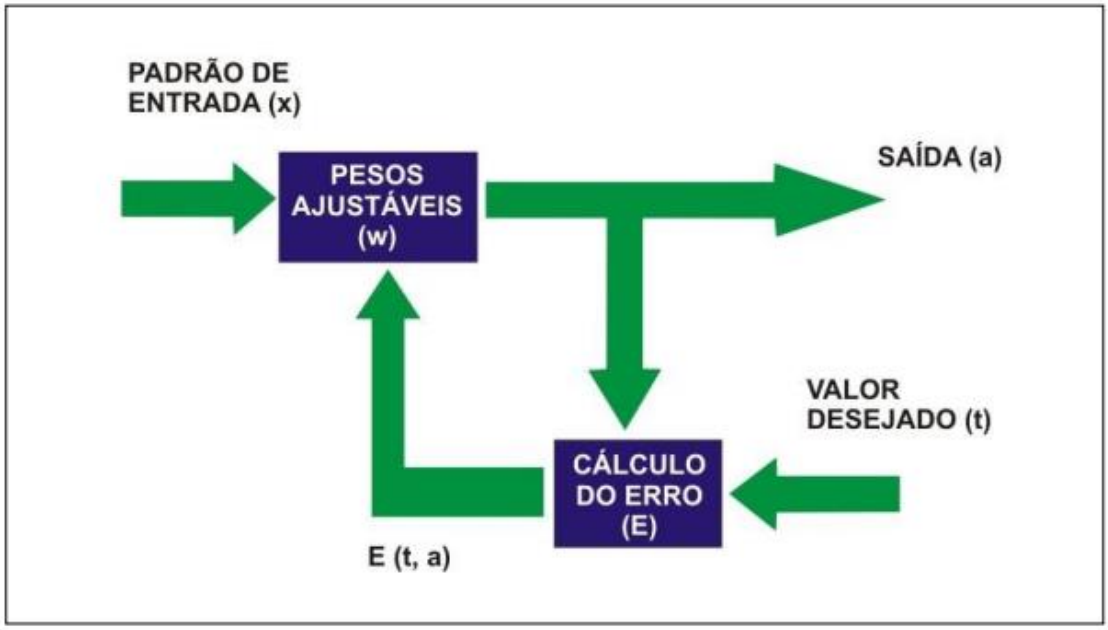
• função de transferência;

• algoritmo de aprendizado.

Existem diferentes tipos de redes neurais artificiais e diferentes maneiras de classificá-las. Talvez a mais importante seja quanto à forma de aprendizado ou treinamento, que pode ser supervisionado ou não-supervisionado.

No aprendizado supervisionado são apresentados à rede padrões de entrada e suas saídas (respostas desejadas). Durante este processo, a rede realiza um ajustamento dos pesos das conexões entre os elementos de processamento, segundo uma determinada regra de aprendizagem, até que o erro calculado em função das saídas geradas pela rede alcance um valor mínimo desejado (MEDEIROS, 1999).

O aprendizado supervisionado (que é o foco deste trabalho) em uma rede neural artificial é ilustrado na Figura 5.3:



No aprendizado não-supervisionado, a rede analisa os conjuntos de dados apresentados e determina algumas propriedades dos conjuntos de dados, “aprendendo” a refletir estas propriedades na sua saída. A rede utiliza padrões, regularidades e correlações para agrupar os conjuntos de dados em classes.

As propriedades que a rede “aprende” sobre os dados pode variar em função do tipo de arquitetura utilizada e da forma de aprendizagem. Por exemplo, Mapa AutoOrganizável de Kohonen, Redes de Hopfield e Memória associativa Bidirecional, são alguns métodos de aprendizado não-supervisionado (MEDEIROS, 1999).

Redes Neurais: Radial Basis Function (RBF)

As redes RBF são modelos de Redes Neurais Artificiais inspirados pelas respostas “localmente sintonizáveis” de alguns neurônios biológicos. Estas células, encontradas em muitas partes dos sistemas nervosos biológicos, respondem a características selecionadas de algumas regiões finitas do espaço dos sinais de entrada (HASSOUN, 1995).

A rede neural RBF apresenta uma estrutura em três camadas em modelo feedforward (ou seja, sem conexões recorrentes entre os neurônios). Esta arquitetura é a mais indicada para tarefas de classificação e predição, porque representa melhor os mapeamentos estáticos de entrada e saída; modelos que utilizam arquiteturas recorrentes (ou seja, redes neurais que possuem conexões de retorno ou feedback) são mais complexas matematicamente e mais apropriadas para problemas dinâmicos, como análises de séries temporais ou aplicações em controle (KLÖSGEN et al., 2002).

A arquitetura utiliza uma função de transferência linear para as unidades de saída e uma função não-linear de transferência (normalmente Gaussiana) para as unidades ocultas.

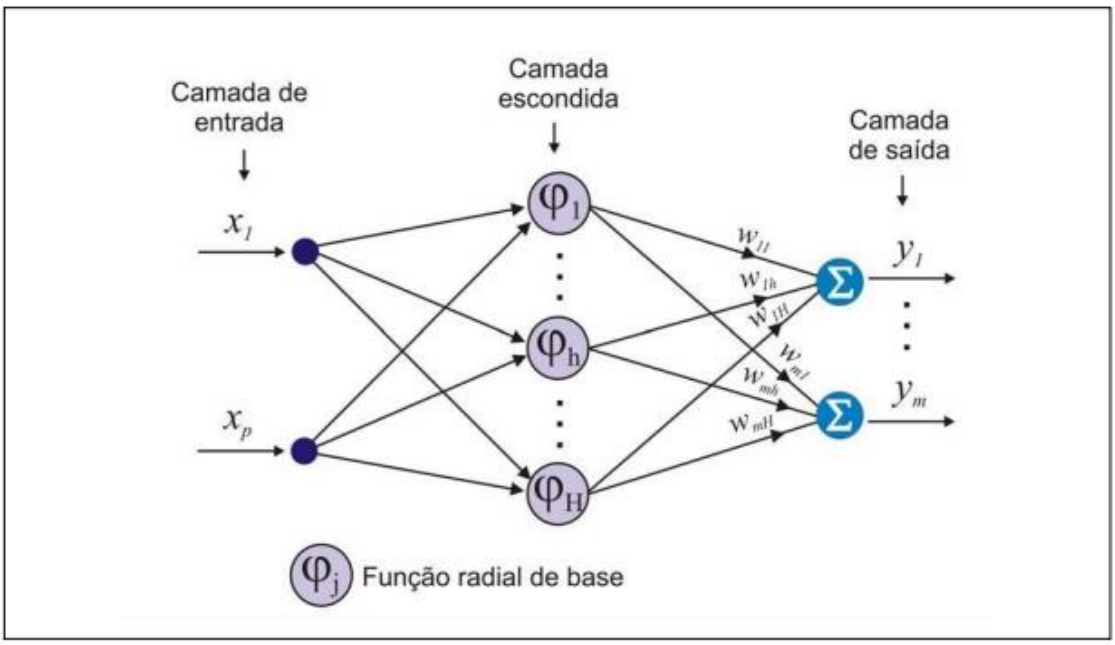
A camada de entrada consiste de n unidades conectadas por conexões com pesos para a camada oculta. Em geral, uma rede RBF pode ser descrita como uma construção global de aproximações para funções utilizando combinações de funções de base centralizadas em vetores de pesos (JAGOTA, 1998). Segundo Fernandes et al. (1999), as redes de funções radiais de base

(redes RBF) têm significativa posição dentro do domínio das redes neurais artificiais.

A principal razão para esse resultado é a simplicidade do processo de treinamento e a eficiência computacional. A estrutura da rede RBF é do tipo múltiplas camadas, o método de treinamento é do tipo feedforward e o treinamento pode ser supervisionado (método aplicado neste trabalho), ou híbrido (combinando um método não-supervisionado com um supervisionado).

A estrutura básica de uma rede RBF é apresentada na Figura 5.4. A primeira camada é a conexão do modelo como o meio. A segunda camada (ou camada escondida) realiza uma transformação não-linear do espaço vetorial de entrada para um espaço vetorial interno que geralmente tem uma dimensão maior (FERNANDES et al., 1999).

A última camada, a camada de saída, transforma o espaço vetorial interno em uma saída, através de um processo linear. Os neurônios da camada escondida são funções radiais de base (FERNANDES et al., 1999).



As funções radiais de base produzem uma resposta significativa, diferente de zero, somente quando o padrão de entrada está dentro de uma região

pequena localizada no espaço de entrada. Cada função requer um centro e um parâmetro escalar. A função que é mais utilizada com função de radial de base é a função de Gauss (FERNANDES et al., 1999).

$$\varphi(v) = \exp\left(-\frac{1}{2\sigma^2} \|v - c\|^2 \right)$$

Assim, uma componente yj do vetor de saída y da rede RBF é caracterizada como:

$$y_j = F_k(x) = \sum_{h=1}^{II} w_{kh} \varphi_h(x_s), \quad \begin{matrix} j=1,2,\ldots,m. \\ s=1,2,\ldots,N. \end{matrix}$$

Substituindo a Equação 5.2 na Equação 5.3:

$$y_j = F_j(x) = \sum_{h=1}^{II} w_{jh} \exp\left(-\frac{1}{2\sigma_h^2} \|x_s - ch\|^2 \right)$$

Onde wjh é o peso sináptico entre o neurônio h da camada escondida com o neurônio j da camada de saída; xs é o s – ésimo vetor de entrada de um conjunto de treinamento X; e ch é o vetor de centro relativo ao neurônio h da camada escondida, definido por:

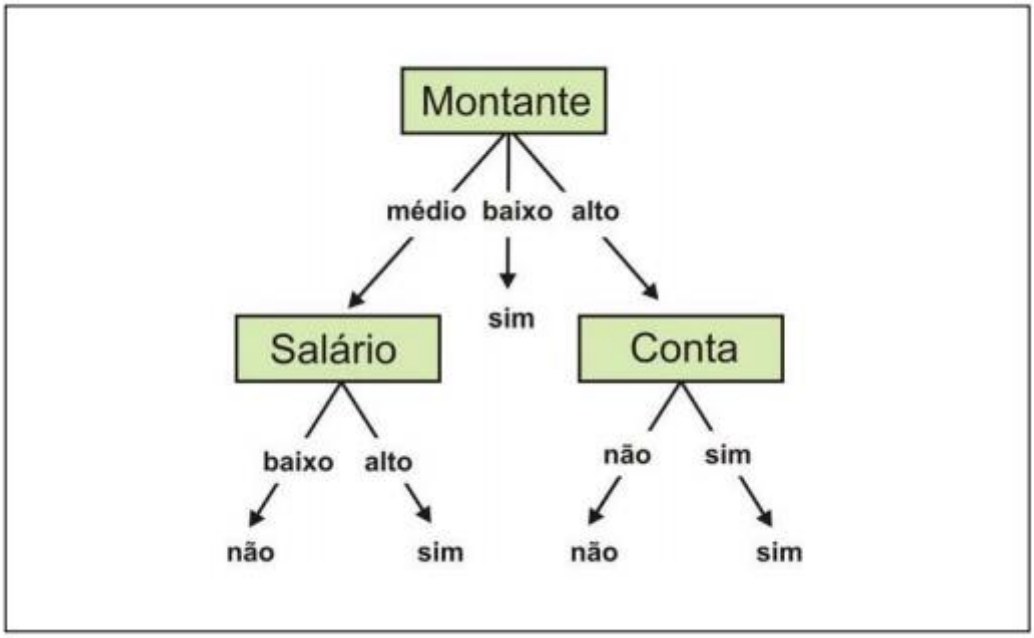
$$c_h = \left[c_{h1}, c_{h2,} \ldots, c_{hj}, \ldots, c_{hp} \right]^T$$

Árvores de decisão

Amplamente utilizadas em algoritmos de classificação, as árvores de decisão são representações simples do conhecimento e, um meio eficiente de construir classificadores que predizem classes baseadas nos valores de atributos de um conjunto de dados (GARCIA, 2000). As árvores de decisão consistem de nodos que representam os atributos, de arcos, provenientes destes

nodos e que recebem os valores possíveis para estes atributos, e de nodos folha, que representam as diferentes classes de um conjunto de treinamento (HOLSHEIMER, 1994).

Uma árvore de decisão tem a função de particionar recursivamente um conjunto de treinamento, até que cada subconjunto obtido deste particionamento contenha casos de uma única classe. Para atingir esta meta, a técnica de árvores de decisão examina e compara a distribuição de classes durante a construção da árvore. Os resultados obtidos após a construção de uma árvore de decisão são dados organizados de maneira compacta, que são utilizados para classificar novos casos (HOLSHEIMER, 1994). A Figura 5.5 apresenta um exemplo de árvore de decisão.



No exemplo da Figura 5.5, são trabalhados objetos que relatam as condições propícias de uma pessoa receber ou não um empréstimo. É considerada a probabilidade do montante do empréstimo ser médio, baixo ou alto. Alguns objetos são exemplos positivos de uma classe sim, ou seja, os requisitos exigidos a uma pessoa, por um banco, são satisfatórios à concessão de um empréstimo, e outros são negativos, onde os requisitos exigidos não são satisfatórios à concessão de um empréstimo. Classificação, neste caso, é a

construção de uma estrutura de árvore, que pode ser usada para classificar corretamente todos os objetos do conjunto (BRAZDIL, 1999).

A partir de uma árvore de decisão é possível derivar regras. As regras são escritas considerando o trajeto do nodo raiz até uma folha da árvore. Estes dois métodos são geralmente utilizados em conjunto. Devido ao fato das árvores de decisão tenderem a crescer muito, de acordo com algumas aplicações, elas são muitas vezes substituídas pelas regras. Isto acontece em virtude das regras poderem ser facilmente modularizadas. Uma regra pode ser compreendida sem que haja a necessidade de se referenciar outras regras (INGARGIOLA, 1996). Com base na árvore de decisão apresentada na Figura 5.5, pode-se exemplificar a derivação de regras.

Dois exemplos de regras obtidas a partir desta árvore são mostrados a seguir:

Se montante = médio e salário =baixo

então classe = não

Se montante = médio e salário =alto então classe = sim

Existem diferentes algoritmos de classificação que elaboram árvores de decisão. Não existe uma regra que determine parâmetros de performance para a definição do melhor algoritmo e, como no caso das redes neurais, cada algoritmo pode apresentar desempenho satisfatório em função do problema em análise. A formação de uma árvore de decisão segue os seguintes passos (BRAGA et al., 2004):

1) associar a partição do nó-raiz ao espaço de objetos;

2) verificar se o nó atual é um nó folha checando se pelo menos um dos seguintes quesitos é verdadeiro:

- todos os objetos contidos na partição do nó atual são da mesma classe;
- todos os atributos de objetos já foram utilizados no teste de algum nó no caminho deste até a raiz;

- a quantidade de objetos na partição do nó atual é inferior ao limite estabelecido (o limite mínimo é 1).
- no caso do nó atual ser uma folha, encerrar a exploração deste.

3) dividir a partição do nó atual segundo um atributo que não foi utilizado em nenhum outro teste sobre atributo no caminho entre o nó atual e o nó raiz;

4) aplicar recursivamente o passo 2 e 3 do algoritmo para cada nó filho do nó atual.

Após a construção de uma árvore de decisão é importante avaliá-la. Esta avaliação é realizada através da utilização de dados que não tenham sido usados no treinamento. Esta estratégia permite estimar como a árvore generaliza os dados e se adapta a novas situações, podendo, também, se estimar a proporção de erros e acertos ocorridos na construção da árvore (BRAZDIL, 1999).

Existem questões a serem superadas em qualquer algoritmo de construção de Árvores de Decisão para que esta seja ótima em quesitos como altura, eficiência de classificação, tempo de construção, entre outros. Alguns destes, que ainda hoje são tema de pesquisa, são listados a seguir (BRAGA et al., 2004):

- escolha da melhor partição para um nó ¾ em geral, por escolha do atributo.
- estratégias para limitação no crescimento da árvore.
- tratamento de valores desconhecidos no conjunto objetos para treino e para teste.
- partições baseadas em características discretas contínuas.

O algoritmo ID3 (Inductive Decision Tree) foi um dos primeiros algoritmos de árvore de decisão, tendo sua elaboração baseada em sistemas de inferência e em conceitos de sistemas de aprendizagem. Proposto por Quinlan (1986), é aplicável para conjuntos de objetos com atributos discretos ou discretizados.

Este algoritmo possui uma implementação simples e um bom desempenho, o que levou a ser este um dos mais populares. Sua estrutura é a

mesma do algoritmo básico apresentado anteriormente; a inovação reside no critério de seleção de atributos para o particionamento, que é o Critério da Entropia. Como características principais do algoritmo ID3 citam-se (BRAGA et al., 2004):

- espaço de busca de hipóteses (árvores) é completo; ou seja, não há o risco de a melhor árvore (target function) não estar neste espaço.
- ID3 não realiza backtracking na busca pela melhor árvore (ou seja, uma vez escolhido um atributo de teste em um nível particular da árvore, ele nunca retrocede a este nível para reconsiderar esta escolha). Por isso, há o risco da solução encontrada corresponder a uma solução ótima local;
- ID3 usa todas as instâncias do conjunto de treino em cada passo da busca devido à seleção de atributos baseada em medidas estatísticas. Com isso, este algoritmo é menos sensível à presença de instâncias erroneamente classificadas ou com atributos sem valores.

Após a popularização do algoritmo ID3, foram elaborados diversos algoritmos, sendo os mais conhecidos: C4.5, CART (Classification and Regression Trees), CHAID (Chi Square Automatic Interaction Detection), entre outros (GARCIA, 2000).

O algoritmo C4.5 é uma extensão do ID3, com alguns aperfeiçoamentos, dos quais pode-se citar principalmente: redução em erros de poda na árvore; implementação de regras de verificação após poda; capacidade de trabalhar com atributos contínuos; capacidade de trabalhar com atributos de valores ausentes e aperfeiçoamento da eficiência computacional.

Uma implementação do algoritmo C4.5 é implementada no classificador J48, disponibilizado pela ferramenta WEKA – e será utilizado no estudo de caso como ferramenta de classificação utilizando a metodologia de árvores de decisão.

Classificadores Bayesianos

Thomas Bayes sugeriu uma regra (regra de Bayes, publicada em 1763) possibilitando que a probabilidade de um evento possa ser dada com base no conhecimento humano, ou seja, em eventos nos quais não se pode medir a freqüência com que ocorrem - a probabilidade pode ser dada com base no conhecimento que um especialista tem sobre o mesmo (HRUSCHKA et al., 1997). A estatística bayesiana passou a ser aplicada em sistemas de Inteligência Artificial (IA) no início dos anos 60. Naquela época, o formalismo da utilização de probabilidades condicionais ainda não estava bem definido (MELLO, 2002).

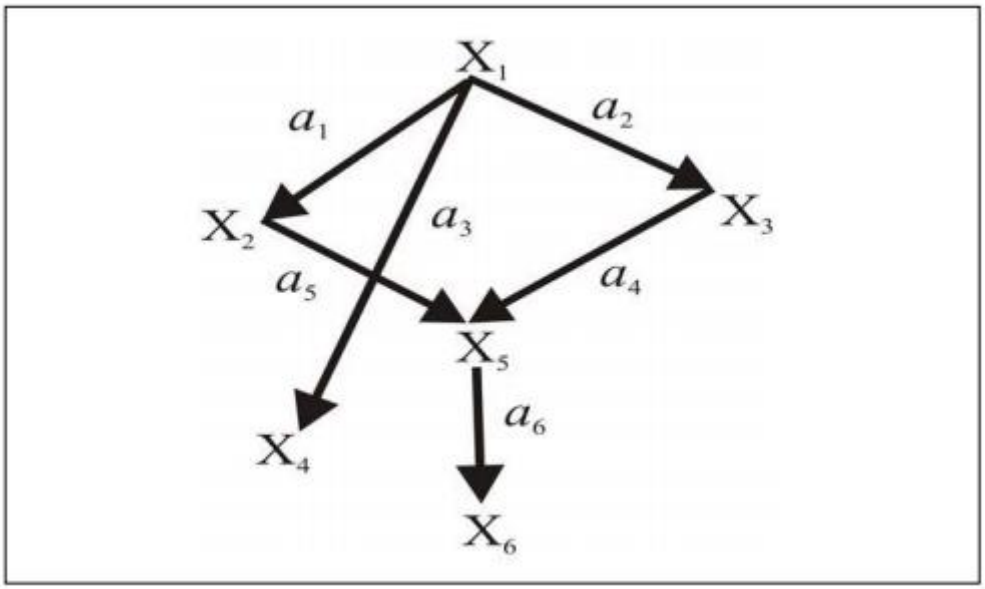
Hruschka et al. (1997) define a probabilidade bayesiana como uma teoria consistente e que permite a representação de conhecimentos certos e incertos sobre condições de incerteza via distribuição conjunta de probabilidades. Tal distribuição conjunta pode ser representada pelo produto de distribuições condicionadas, como, por exemplo:

$P(X1,X2,X3,X4,X5,X6)=P(X6|X5)\ P(X5|X2,X3)\ P(X2|X1)\ P(X4|X1)\ P(X3|X1)\ P(X1)$

Uma variável é condicionada a uma ou mais variáveis numa relação causal. Uma distribuição pode ser representada por um grafo orientado. No grafo, cada nó representa uma variável do modelo, e os arcos ligam as variáveis que estão em relação direta causa / efeito. Por exemplo, se houvesse uma pesquisa com pessoas que passam pela rua, que perguntasse: que dia é hoje? Sem dúvida, a resposta da maioria seria a mesma. Isso porque a maioria das pessoas segue um mesmo calendário e isso causa o fato das respostas serem aproximadamente iguais. Sendo assim, caso deseja-se saber qual será a resposta do próximo entrevistado, não é necessário observar todas as respostas anteriores; basta observar a causa, ou seja, o fato do calendário ser utilizado pela maioria da população. A esta estrutura gráfica, com a quantificação de crença nas variáveis e seus relacionamentos, dá-se o nome de Redes Bayesianas (RB) ou Redes Causais (MELLO, 2002).

Uma rede bayesiana é um gráfico direcionado acíclico (ou DAG - Directed Acyclic Graph) com uma distribuição de probabilidades condicionais para cada

nó. Cada nó x representando uma variável de domínio, e cada arco representando uma dependência condicional entre os nós (CHENG et al., 1999). No aprendizado em redes bayesianas, usa-se o nó para representar atributos dos conjuntos de dados, como representado na Figura5.6.



Para verificar se um grafo DAG é uma rede bayesiana, precisa-se verificar se cada variável x do grafo deve ser condicionalmente independente de todos os nós que não são seus descendentes exceto seus pais (PERL, 1988). Esta condição permite reduzir consideravelmente o esforço computacional, porque existe uma explosão combinatória no cálculo da distribuição conjunta das probabilidades.

Para reduzir o esforço computacional, basta explorar as distribuições das relações entre as variáveis do problema (PERL, 1988). Graças a esta distribuição foram desenvolvidos vários algoritmos de propagação de crença em RB, os quais permitiram, sobre uma rede, propagar o conhecimento de novos fatos. Propagação de crenças corresponde em estabelecer um procedimento que, explorando a conectividade da rede, permita a determinação das distribuições de probabilidades das variáveis, objetivo do diagnóstico, condicionadas aos valores das variáveis que representam evidências. Assim, o conhecimento incerto pode ser atualizado de forma simples e clara, mantendo a consistência e a confiabilidade.

As redes bayesianas podem ser visualizadas de duas maneiras:

▪ como uma estrutura que codifica um grupo de relacionamentos de independência condicional entre os nós, conforme um conceito chamado de d-serapação (quando uma RB implica mais independência condicional do que geralmente aquelas independências envolvidas com os pais de um nó.). A idéia é que uma estrutura de RB possa ser instruída pelo mecanismo de aprendizagem com a independência condicional entre os nós da rede. Usando alguns testes estatísticos, pode-se encontrar a relação de independência condicional entre os atributos e usar o relacionamento como forma para construção de uma RB;

▪ como uma estrutura que codifica a união distributiva dos atributos. Então uma RB pode ser usada como um classificador que dá a posterior probabilidade distributiva do nó de classificação, dados os valores de outros atributos.

Com a RB pode-se representar problemas do mundo real em que existam relações de causa e conseqüência entre as variáveis. Isso motiva o uso desses tipos de redes porque os seres humanos têm uma certa necessidade de representar o conhecimento utilizando fatos em forma de relacionamentos causais (HRUSCHKA et al., 1997). As RB's também possuem a possibilidade de realizar o aprendizado a partir dos dados. Nesse aprendizado, oferece-se uma amostra e o sistema, através de um algoritmo, gera uma estrutura que melhor se adapta aos dados do problema (HECKERMAN, 1995). Existem vários algoritmos e métodos para o aprendizado de RB's a partir dos dados, sendo que cada um se adapta melhor a uma determinada classe de problema (MELLO, 2002).

O Classificador Naive Bayes

As redes bayesianas têm sido usadas em processos de classificação há muitos anos. Quando sua estrutura apresentar uma classe como nó pai de todos os outros nós e nenhuma outra conexão é permitida, torna-se ideal para os processos de classificação. Esta estrutura é comumente chamada de redes Naive Bayes, que é um caso especial de redes probabilísticas ou redes bayesianas (MELLO, 2002). O princípio básico de classificadores bayesianos

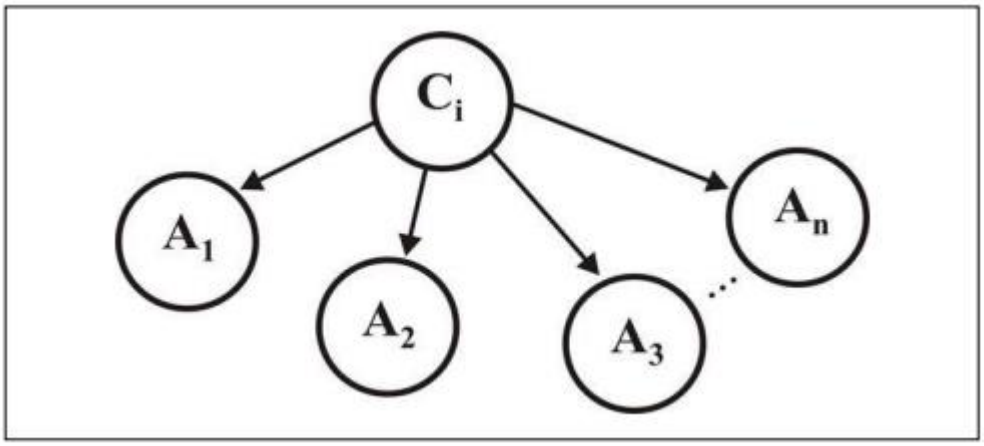
está fundamentado na teoria da probabilidade bayesiana (DUDA et al., 2000). Os classificadores bayesianos são capazes de encontrar regras que respondem a perguntas do tipo:

▪ qual a probabilidade de se jogar tênis dado que o dia está ensolarado, com temperatura quente, umidade alta e vento fraco? Em termos probabilísticos, essa pergunta equivale a P(JOGAR TÊNIS = Sim | [Ensolarado, Quente, Alta, Fraco]);

▪ qual a probabilidade de NÃO se jogar tênis dado que o dia está ensolarado, com temperatura quente, umidade alta e vento fraco? Em termos probabilísticos essa pergunta equivale a P(JOGAR TÊNIS = Não | [Ensolarado, Quente, Alta, Fraco]).

Na representação de gráfico acíclico de uma rede bayesiana, cada nó representa um atributo (interpretado como uma variável randômica), que é usado para descrever um domínio de interesse, e cada ligação representa uma dependência entre os atributos. O gráfico mostra uma particular junção de probabilidade distributiva, onde cada nó da rede representa uma probabilidade condicional distributiva entre os valores dos atributos (MELLO, 2000).

Uma rede bayesiana como classificador Naive Bayes apresenta o gráfico em forma de estrela, no qual o centro da estrela é a classe que será classificada. Como pode ser observado na Figura 5.7, os atributos formam as pontas da estrela (A1 a An). A única conexão possível é cada atributo com a classe (Ci) . Nenhuma outra conexão é permitida na rede Naive Bayes (MELLO, 2000).



A rede Naive Bayes tem sido usada por pesquisadores, em classificações há muitos anos, por apresentar características vantajosas sobre outros tipos de classificadores, tais como:

▪ facilidade na construção de seu algoritmo: pela simplicidade do seu algoritmo, estimulou pesquisadores a aplicar este método em muitas ferramentas (CHEN et al., 1997);

▪ o processo de classificação é muito eficiente quando os atributos são independentes entre si: em situações onde os atributos não são correlacionados, o classificador Naive Bayes sobressai surpreendentemente sobre muitos sofisticados classificadores. Esta característica é rara na prática de aprendizagem. Isso ocorre porque a rede Naive Bayes apresenta uma limitação nas ligações entre os nós (conforme Figura 5.7);

▪ é rápido na aprendizagem e predição: seu tempo de aprendizagem e de predição é linear independentemente do número de exemplos. Pesquisadores têm desenvolvido trabalhos comparando e ressaltando o bom desempenho do classificador Naive Bayes em relação a outros modelos de classificação complexos - um comparativo é descrito por Friedman et al. (1997).

É interessante citar que existem fatos que podem gerar problemas nas predições do classificador Naive Bayes, formando algumas limitações para o algoritmo, como por exemplo (MELLO, 2000):

▪ trabalhar com valores com casas decimais. Os erros causados por arredondamentos podem causar variações na predição;

▪ os exemplos devem apresentar atributos com independência condicional, caso contrário, o Naive Bayes se torna inviável.

Apesar desses fatores, o classificador Naive Bayes ainda se torna eficiente para utilização em aplicações que envolvem predição. Sua facilidade de implementação e seu desempenho colocam o algoritmo como um dos mais

citados classificadores nas pesquisas na área de Inteligência Artificial (MELLO, 2000).

SVM (Support Vector Machines)

Apesar de relatos dos anos 60 sobre a técnica de SVM, foi em 1992 que um primeiro artigo foi apresentado por Vladimir Vapnik, Bernhard Boser e Isabelle Guyon [5]. Apesar de ser uma técnica nova, tem chamado muita atenção pelos seus resultados: obtém altos índices de assertividade, permite modelar situações não-lineares complexas gerando modelos de simples interpretação, pode ser usada para relações lineares e não-lineares, entre outros. É utilizado tanto para tarefas de classificação quanto de predição. Atualmente um dos problemas da técnica de SVM é o tempo utilizado no aprendizado. Muitas pesquisas tem se concentrado neste aspecto.

Classificação por Regras de Associação (Classification by Association Rule)

Recentemente, as técnicas de Regras de Associação estão sendo usadas para a classificação. A ideia geral é buscar por padrões de associações fortes entre os itens (utilizando-se do conceito de frequência) e as categorias. Basicamente consiste em dois passos: primeiro, os dados de treinamento são analisados para que se obtenha os itens mais frequentes. Em seguida, estes itens são usados para a geração das regras. Alguns estudos demostraram que esta técnica tem apresentado mais assertividade do que algoritmos tradicionais, como o C4.5. Alguns exemplos de algoritmos de classificação são: CBA (Classification-Based Association) , CMAR (Classification based on Multiple Association Rules) [40] e CPAR . mostra uma nova abordagem chamada de CARM (Classification Association Rule Mining).

Aprendizado Tardio (Lazy Learners)

As técnicas de classificação descritas até agora usam um conjunto de dados de treinamento para aprender a classificar um novo registro. Assim, quando são submetidas a um novo registro elas já estão prontas, ou seja, já aprenderam. Existe, no entanto, uma outra categoria de métodos, que somente realizam esse aprendizado quando solicitado para a classificação de um novo

registro. Neste caso, o aprendizado é considerado tardio. Apesar de necessitar de um tempo menor de treinamento, esses métodos são muito dispendiosos computacionalmente, pois necessitam de técnicas para armazenar e recuperar os dados de treinamento. Por outro lado, esses métodos permitem um aprendizado incremental. O algoritmo conhecido como kNN (k - Nearest Neighbor), descrito na década de 50, só tornou-se popular na década de 60, com o aumento da capacidade computacional. Basicamente, esse algoritmo armazena os dados de treinamento e quando um novo objeto é submetido para classificação, o algoritmo procura os k registros mais próximos (medida de distância) deste novo registro. O novo registro é classificado na classe mais comum entre todos os k registros mais próximos. No algoritmo chamado de Case-Based Reasoning (CBR), ao invés de armazenar os dados de treinamento, ele armazena os casos para a solução dos problemas. Para a classificação de um novo objeto, a base de treinamento é analisada em busca de uma solução. Caso não encontre, o algoritmo sugere a solução mais próxima. Esse algoritmo tem sido bastante utilizado na área de suporte aos usuários, Médica, Engenharia e Direito.

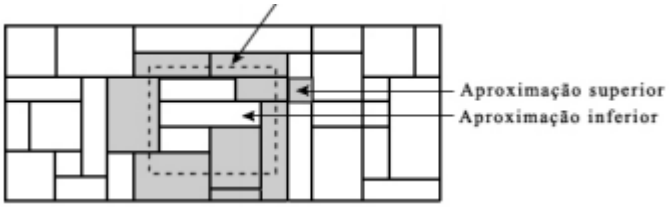
Algoritmo Genético (Genetic Algorithm)

A ideia dos algoritmos genéticos segue a teoria da evolução. Geralmente, no estágio inicial uma população é definida de forma aleatória. Seguindo a lei do mais forte (evolução), uma nova população é gerada com base na atual, porém, os indivíduos passam por processos de troca genética e mutação. Este processo continua até que populações com indivíduos mais fortes sejam geradas ou que atinga algum critério de parada.

Conjuntos Aproximados (Rought Set)

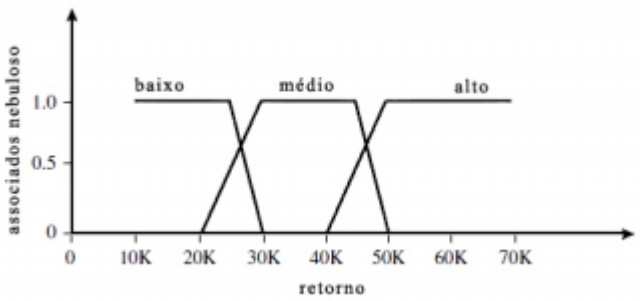
É uma técnica que consegue realizar a classificação mesmo com dados impreciso ou errados e é utilizada para valores discretos. A ideia geral destes algoritmos é a de classe de equivalência: eles consideram que os elementos de uma classe são indiscerníveis e trabalham com a ideia de aproximação para a criação das categorias. Por exemplo, uma estrutura (chamada Rought Set [25]) é criada para uma classe C. Esta estrutura é cercada por dois outros conjuntos de aproximação (chamados de baixo e alto). O conjunto de baixa aproximação

de C contém os registros que certamente são desta classe. O conjunto de alta aproximação contém os registros que não podem ser definidos como não pertencentes à classe C. Um novo registro é classificado mediante a aproximação com um destes conjuntos. Busse [24] faz uma comparação do algoritmo MLEM2 (Modified Learning from Examples Module, version 2)) com duas variações. Uma representação dos conjuntos aproximados pode ser vista na figura 10.



Conjuntos Nebulosos (Fuzzy Set)

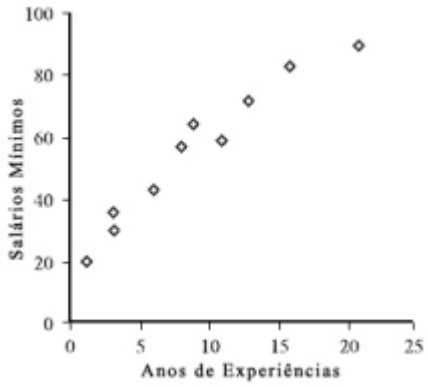
A classificação baseada em regras apresenta um problema relacionado às variáveis contínuas. Elas necessitam de um ponto de corte bem definido, o que às vezes é ruim ou impossível. Por exemplo, SE salario > 4.000 ENTÃO credito = ok. Porém, registros com salário de 3.999 não serão contemplados. Proposta por Lotfi Zadeh em 1965, a ideia dos conjuntos Fuzzy é de que, ao invés de realizar um corte direto, essas variáveis sejam discretizadas em categorias e a lógica nebulosa aplicada para definição dos limites destas categorias. Com isso, ao invés de se ter as categorias com limites de corte bem definido, tem-se um certo grau de flexibilidade entre as categorias. Na figura 11 pode-se ver um exemplo de um conjunto nebuloso.



Regressão Linear

As regressões são chamadas de lineares quando a relação entre as variáveis preditoras e a resposta segue um comportamento linear. Neste caso,

é possível criar um modelo no qual o valor de y é uma função linear de x. Exemplo: y = b + wx. Pode-se utilizar o mesmo princípio para modelos com mais de uma variável preditora. Na figura 12 tem-se um exemplo de uma regressão linear.



Regressão Não-Linear

Nos modelos de regressão não-linear, a relação entre as variáveis preditoras e a resposta não segue um comportamento linear. Por exemplo, a relação entre as variáveis pode ser modelada como uma função polinomial. Ainda, para estes casos (Regressão Polinomial), é possível realizar uma conversão pra uma regressão linear. Outros modelos também são encontrados na literatura: Logistic Regression, Poisson Regression e Log-Linear Models.

ESTUDO DE CASO I

Classificação e Predição de Preços de Imóveis a Partir de Dados Estruturados e Não Estruturados

CAPÍTULO 1

Introdução

A avaliação imobiliária, que é o processo de estimar o preço de imóveis, é crucial para ambos compradores e vendedores como uma base para a negociação e transação. O mercado imobiliário tem um papel vital em todos os aspectos da nossa sociedade contemporânea [QY17]. Com o crescimento da geração de dados online sobre imóveis, o caminho mais intuitivo tomado para a análise desses foi o de estimar esse preço através dos atributos desses imóveis, ou seja, os dados já estruturados, como quantidade de quartos, localização, área interna, etc. Porém, outra forma de talvez aumentar a acurácia e precisão de um algoritmo que faça essa estimativa seria utilizar em conjunto com os dados estruturados, dados não-estruturados, neste caso, as descrições dos imóveis, comentários sobre ele e avaliações de usuários.

Todos esses são exemplos de dados não-estruturados em forma de texto. Que seriam tratados de forma a inferir características adicionais aos imóveis que poderiam influenciar no seu preço, e dessa forma aumentando a precisão do algoritmo. A análise de dados não-estruturados, mais especificamente de textos, para a predição de preços de produtos é uma tendência que vem aumentando nos últimos anos. Uma competição promovida pela Kaggle e a Mercari [Mer18], neste ano, teve como foco a construção de um algoritmo que sugeria automaticamente o preço correto de produtos [Mer18], sendo fornecidos dados não-estruturados em forma de textos de descrições escritos por usuários com informações como marca, categoria do produto e condições.

Utilizando essas duas formas de avaliação imobiliária através de algoritmos de análise de dados estruturados e não-estruturados, poderemos comparar a utilização das duas formas de avaliação e como um interfere na precisão e acurácia do outro quando utilizadas em conjunto. Desse modo pretende-se criar uma ferramenta que consiga utilizar os dois métodos separadamente, assim como também em conjunto e compará-los.

CAPÍTULO 2

Fundamentos

Para a construção de um modelo de regressão de qualidade vários processos de tratamento, ajuste e pre-processamento dos dados devem ser feitos a priori. Esses processos são essenciais para garantir o melhor desempenho e qualidade de um modelo de regressão. Neste capítulo 2, iremos abordar alguns desses processos que involvem dados estruturados e não-estruturados. Aprofundando mais no pré-processamento textual.

Pré-processamento de dados

Antes de utilizar qualquer tipo de dado para construção de um modelo estatístico, aplicação de algoritmo de aprendizagem supervisionada ou mesmo uma análise dos dados, deve haver um tratamento desses dados para que a corretude, qualidade e desempenho dessas tarefas. A extração dos dados pode vir de várias fontes: bancos de dados, tabelas excel, html, etc. E muitas vezes quando extraídos alguns problemas devem ser resolvidos. Dentre estes estão a decteção e limpeza de dados errônios ou mesmo irrevalentes, como tanto outliers - observações ou medidas suspeitas por ser muito menor ou maior do que a maioria das observações coletadas. [DC10] Nessa seção 2.1, iremos introduzir o conceito de dados estruturados e alguns dos problemas que podem ser resolvidos no pré-processamento para a construção de modelos melhores. Iremos também introduzir o conceito de dados não-estruturados e abordar a extração de textos a partir de páginas da web e o seu pré-processamento.

Dados estruturados

Dados estruturados são caracterizados por serem dados que possuem uma organização, dados que podem ser facilmente acessados diretamente através de tabelas ou bancos de dados. Nesta seção iremos abordar alguns processos de pré-processamento de dados estruturados.

Limpeza de dados

Limpeza de dados é um processo de pré-processamento de dados definido por três estágios envolvidos em um ciclo repetitivo: triagem, diagnóstico e edição (ou remoção) de anormalidades. [GJ13] A fase de triagem tem como objetivo verificar outliers, inconsistências, padrões anormais e a falta ou excessão de informação. Uma estratégia para detecção de anomilias é inserir os dados em um tabela onde podem ser visualizados e verificados de diversas maneiras. Inicialmente, uma análise de consulta dessas tabelas pode ser realizada para verificar se há algum dado discrepante, irrelevante ou faltante, pode ser feita. Como por exemplo, um conjunto de dados que descreve imóveis ter um valor de venda do imóvel menor que zero, ou até mesmo muito baixo em comparação aos outros. Após a análise através de consultas, uma análise estatística através gráficos como boxplot, histogramas e gráficos de dispersão pode ser feita para que a visualização dessas anormalidades seja mais evidente para o cientista de dados. A fase de diagnóstico tem como objetivo clarificar a natureza dos problemas, padrões e estatísticas identificados na fase de triagem. Nessa fase, os diagnósticos podem verificar facilmente um dado impossível (como o valor de venda negativo), suspeito(como valor de venda muito baixo) ou faltante (onde simplesmente não há valor no conjunto de dados para aquela instância). A fase de tratamento tem como objetivo resolver os problemas analisados nas fases anteriores, tendo como opção corrigir, deletar ou mesmo ignorar os dados anormais. Para dados impossíveis ou faltantes, quando possível encontrar os dados corretos, uma correção deve ser feita, porém se não houver como encontrá-los, é preferível uma deleção. O julgamento para resolução desses problemas varia bastante caso a caso.

Dados não estruturados

Dados não estruturados são conjunto de dados que não possuem uma organização definida. Dados em que as informações não podem ser extraídas diretamente e por isso exigem uma aproximação diferente quanto ao pré-processamento. Nesta seção iremos analisar algumas técnicas de extração e pré-processamento de dados não estruturados, mais especificamente dados textuais.

Extração de texto a partir de HTML

A extração de dados não-estruturados (ou semi-estruturados) na maioria das vezes tem como foco as páginas em si, no caso dos textos, o conteúdo da página como um todo. Porém, há também técnicas de mapeamento ou estruturamento desses dados.

A extração pode tanto ser na força bruta usando expressões regulares para extrair informações [ABT16] de uma tag específica ou mesmo de várias tags, quanto abordagens mais sofisticadas. Uma dessas abordagens é o DOM (Document Object Model) que é um padrão para criação e manipulação de representações de HTML (e XML).

Analisando e transcrevendo uma página HTML para uma DOM tree (ou árvore DOM) não só conseguimos extrair grandes unidades lógicas, como também links específicos dentro da árvore. [GKNG03] A partir das DOM trees, modelos de estruturação para esses dados podem ser construídos e assim o pre-processamento deles se transforma no de dados estruturados.

Outra abordagem, que foi a utilizada no projeto, é a de extração textual completa da página. Ou seja, são indentificadas as tags HTML, que são ignoradas, deixando assim somente o texto cru contido em cada página. O texto então é tratamento de forma a transformar cada palavra contida na página em um atributo do conjunto de treinamento. Esses atributos passam por processos de rankeamento e relevancia que serão melhor descritos na próxima subseção.

Técnicas para tratamento de texto

As técnicas de tratamento de texto que iremos abordar abragem o campo de tokenização e de seleção de atributos. A tokenização é um processo de análise do texto pre-processado ou transformado em texto cru para que a partir dele sejam definidas palavras. Esse processo vai definir dentro do texto o que é uma palavra e em qual classe sintática aquela palavra pertence. [GG94] O texto é tratado como uma string única que será divida em pequenas unidades e cada uma delas será interpretada e designada um classe sintática. Além disso, a tokenização também define o que são sentenças e onde elas começam e terminam, verificando pontuações no texto. Após a tokenização, a seleção de atributos vai filtrar os tokens do texto para selecionar atributos a partir deles. O processo de seleção consiste em filtrar tokens que representam palavras de pouco uso e muito uso. Filtrando assim tokens de nomes próprios e preposições. E após esse processo há a seleção de tokens que serão classificados como atributos para o modelo. Existem várias abordagens para rankear a importancia desse atributos sendo as mais conhecimento tf (term frequency) e idf(inverse document frequency), que muitas vezes são usados combinadamente.

A frequencia do termo ou TF, simplesmente faz uma contagem de quantas vezes aquele termo (ou token) aparece em um documento (ou texto). Já o inverso da frequencia no documento é incorporado ao tf (por isso são muitas vezes utilizados em conjunto) para diminuir a importância de termos que tem uma frequencia muito grande. [LPJ02] Após a seleção de atributos e o rankeamento da importancia das palavras no documento ele pode ser utilizado como conjunto de dados para um modelo de aprendizado supervisionado. Porém, outros ajustes como limitação de quantidade de atributos podem ser feitos para melhorar o desempenho do conjunto.

Aprendizado supervisionado

Aprendizado supervisionado é uma tecnica de aprendizagem de máquina que consiste em criar um modelo com uma função que mapea uma entrada à uma saída, de acordo com um conjunto de entradas e saídas já conhecido. Esse conjunto é chamado de conjunto de treinamento. O conjunto de treinamento é analisado pelo algoritmo que cria uma função que irá mapear novas entradas

para saídas corretas, de acordo com ele. Ou seja, no aprendizado supervisionado, para cada entrada do conjunto de treinamento xi ,i = 1,...,n existe uma medida associada yi que é avaliada pelo algoritmo. Assim, um modelo que se encaixe nas condições dadas pelo conjunto de treinamento é criado com o intuito de prever valores de futuras observações ou de entender melhor o relacionamento entre entradas e saídas do algoritmo. [Kot07]

Há dois tipos de modelos de aprendizado supervisionado, sendo esses o modelo de classificação e o de regressão. Geralmente associamos os problemas quantitativos a regressão, enquanto os qualitativos são associados a classificação. Porém, a diferença entre os modelos não é tão simples assim. [Kot07] O modelo de classificação tem como objetivo criar uma função que, a partir de um conjunto de treinamento rotulado, consiga definir a que classe uma nova entrada pertence. Nesse caso, o conjunto de atributos dos dados de treinamento são avaliados e sua classe (ou rótulo) é verificada. A função aprende que características levar em conta quando vai avaliar uma nova entrada para assim rotulá-la. [Spi94] O modelo de regressão tem um objetivo parecido com o de classificação, porém a função criada pelo modelo tem que conseguir prever um valor associado a uma entrada. No caso, ao invés de tentar prever um classe a partir dos atributos do conjunto de treinamento, o algoritmo tenta prever um possível valor para novas entradas a partir desses atributos.

A função aprende a partir do conjunto de treinamento que valor deve atribuir para cada nova entrada, de acordo com os valores presentes no conjunto. Para ambos os casos, após o processo de construção e aperfeiçoamento da função do modelo, chamado de treinamento, a função é validada a partir de um conjunto de testes. Essa validação pode ser verificada a partir das métricas de avaliação. Nesta seção, iremos abordar alguns algoritmos de aprendizado supervisionado, focando nos que envolvem regressão, e suas métricas de avaliação.

Algoritmos de aprendizado supervisionado

Nesta subseção iremos abordar alguns algoritmos de aprendizado supervisionado especificamente ligados a regressão. Todos os algoritmos foram utilizados nesse trabalho.

Random Forest

O algoritmo de Random Forest é um algoritmo baseado em um outro método chamado Decision Tree (ou Árvore de Decisão). Árvore de Decisão é uma algoritmo que utiliza uma estratificação do conjunto de dados em formato de árvore. É uma forma de mapeamento de decisões em que cada folha da árvore é um possível resultado da regressão. Ou seja, de acordo com uma serie de valores escolhidos para os atributos, temos um resultado final: a folha da árvore. Portanto, várias possibilidades de valores são avaliados para os atributos e o valor de cada folha representa o resultado das escolhas dos valores anteriores. O Random Forest é classificado como um algoritmo de "aprendizado em conjunto"ou ensemble learning, portanto utiliza várias Árvores de Decisão em conjunto com uma seleção aleatória de partes do conjunto de dados de treinamento para compor essas árvores. Para cada árvore, um subconjunto de treinamento aleatório com uma seleção de atributos aleatória (também seccionando os atributos dos dados) é criado e executado e ao fim uma média dos resultados é feita para cobrir os valores que não foram executados. Além disso, o Random Forest consegue avaliar a relevancia dos atributos sobre o conjunto de treinamento, dado que as árvores que tiverem os atributos mais relevantes terão resultados melhores e o algoritmo pode ir se ajustando para selecionar esses atributos para a avaliação do conjunto de testes.

Support Vector Machine – Regression

O Máquina de Vetor Suporte, Support Vector Machine ou SVM é um algoritmo muito usado para classificação e também tem uma versão conhecida para regressão o SVR (Support Vector Regression). O SVM divide o espaço do conjunto de dados através vetores de suporte de forma a deixar os pontos de classes diferentes com uma distancia considerável. O vetor que tem a maior margem é escolhido entre os demais para definir as classes. A margem é

definida pelo ponto do espaço que está mais perto da reta. Portanto, os pontos mais distantes da margem teriam sua classe mais fortemente definida, enquanto os que estão mais próximos não tem uma definição tão precisa. O SVR funciona similarmente ao SVM, só que ao invés de classificar os pontos do espaço, queremos estimar um valor numérico. Os vetores suporte são definidos no espaço do conjunto de treinamento de forma a tentar estimar os valores ideiais para os resultados. [Bha18] Após isso uma margem (epsilon) é adicionada ao vetor. Essa margem funciona como uma margem de erro para o valor esperado definido pelo vetor. Portanto os pontos que serão levados em consideração serão os pontos que estarão dentro da margem de erro.

XGBoost

O XGBoost ou eXtreme Gradient Boosting é uma implementação de árvores de decisão de aumento de gradiente com foco em rapidez e perfomace. [Bro16] O XGBoost está sendo fortemente usado em competições e desafios de aprendizado de máquina e big data por sua rapidez. A implementação do algoritmo foi feita para ter a maior eficiência possível no uso de recursos de memória e processamento. Assim tendo as melhores condições possíveis para o treinamento de um modelo.

Métricas de avaliação de um modelo de regressão

Nesta subseção iremos falar sobre algumas métricas de avaliação dos algoritmos de regressão já descritos. Todas essas métricas foram utilizadas neste trabalho. Iremos definir para as equações seguintes y = valor real e ˆy = valor esperado:

ERRO MÉDIO QUADRADO (MSE)

O erro médio quadrado calcula a média dos erros do modelo ao quadrado, com isso, os valores maiores terão mais importância do que os menores. Já que serão levados em consideração os valores ao quadrado. O calculo pode ser descrito pela seguinte equação:

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

ERRO MÉDIO ABSOLUTO (MAPE)

O erro médio absoluto calcula a porcentagem da média dos erros do modelo em valor absoluto, portanto é aplicada uma modulação à subtração. O peso de importância aos erros é dados de maneira linear. O calculo pode ser descrito pela seguinte equação:

$$MAPE = \frac{100}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

ERRO MÉDIO DO LOGARITMO DA DIFERENÇA (MLE)

O erro médio do logaritmo da diferença calcula a média dos logaritmos dos erros. O propósito do uso é para não penalizar discrepancias muito grandes em valores de erros no conjunto. O calculo pode ser representado pela seguinte equação:

$$MLE = \frac{1}{n}\sum_{i=1}^{n}((\log y_i) - (\log \hat{y}_i))$$

COEFICIENTE DE DETERMINAÇÃO (R2)

O coeficiente de determinação é uma métrica de avaliação de regressão que mede quão perto o valor esperado está de se encaixar no modelo. E pode ser descrito como a porcentagem de variação da variavel resultante do modelo ou mesmo:

$$R2 = \frac{V_r}{V_t}$$

Onde V_r = Variação da variavel resultante e V_t = Variação total.

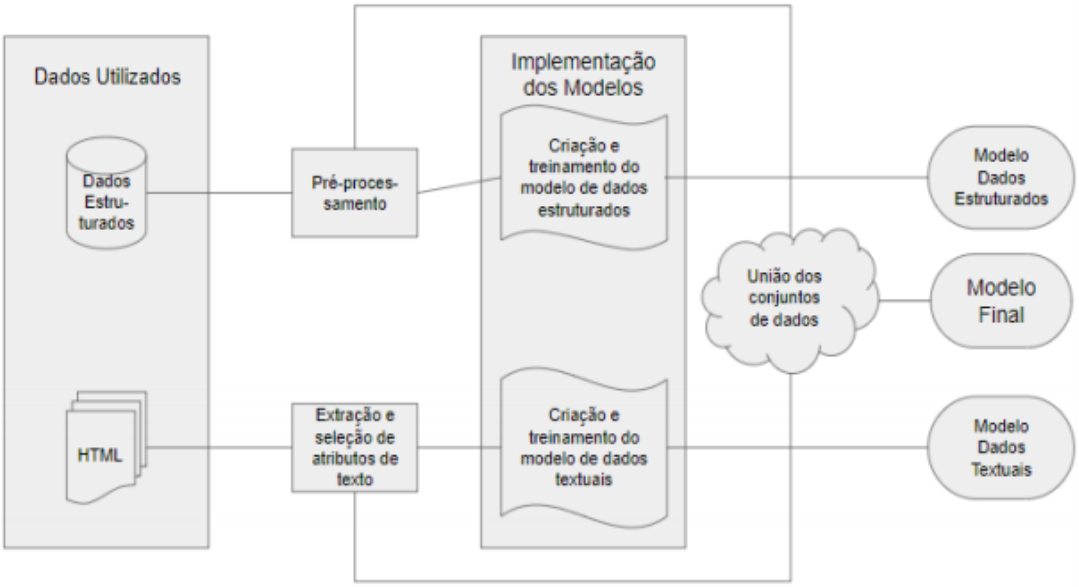
CAPÍTULO 3

IMPLEMENTAÇÃO DO REGRESSOR

Como mencionado anteriormente, esse trabalho tem como objetivo construir um regressor que faça a predição de preços de imóveis a partir de dados estruturados e dados textuais, nãoestruturados, para que seja possível fazer uma comparação do desempenho entre esses tipos de dados, como também em relação a combinação dos dois tipos.

Ou seja, podermos ao fim definir qual o impacto dos dados textuais na definição do preço do imóvel pelo regressor. Esse tipo de avaliação de dados textuais na predição de preços tem sido uma tendencia que vem crescente nos últimos anos. Como já citamos anteriomente, a Kaggle [Mer18] em conjunto com outras empresas vem fazendo desafios e competições que abordam esses temas.

Neste capítulo iremos descrever o processo utilizado para a implementação desse regressor. Descrevendo como os dados foram recebidos e pre-processados, que algoritmos de regressão e técnicas de apredizagem de máquina foram utilizados, como também as dificuldades encontradas na implementação. A figura 3.1 descreve o pipeline da implementação:



Dados Utilizados e Pré-processamento

DADOS UTILIZADOS

Os dados utilizados foram obtidos a partir de um coletor focado no domínio de imóveis, que coleta informações de varios dados da web brasileira. Foram escolhidos a partir da extração, dados de sites de imóveis de 3 cidades brasileiras: Rio de Janeiro (RJ), São Paulo (SP) e Porto Alegre (POA). Para cada uma dessas cidades foram extraídos cerca de dez mil imóveis para compor o conjunto de dados do experimento. A tabela 3.1 descreve a distribuição dos dados.

Status	RJ	SP	POA
Antes do Pré Processamento	7128	10000	10000
Após o Pré Processamento	5208	7304	9489

Os dados estruturados foram recebidos já no formato csv com 8 colunas. Sendo essas colunas: "price"representando o preço, "latitude", "longitude", "bedrooms"representando a quantidade de quartos no imóvel, "bathrooms"representando a quantidade de banheiros no imóvel, "area"representando a área em m², "pkspaces"representando a quantidade de vagas de estacionamento do imóvel, "ensuites"representando a quantidade de suítes presentes no imóvel, "timestamp"representando o horário ao qual o site foi visitado, "type"representando o tipo do imóvel, sendo casa ou apartamento, "operation"representando a operação a qual o imóvel estava sendo ofertado, sendo venda ou aluguel e "url"representando a url do site no qual os dados do imóvel foram extraídos.

Os dados não-estruturados foram recebidos como as páginas HTML presentes na coluna "url"dos dados estruturados. Todas as páginas foram baixadas e adicionadas ao conjunto de dados não-estruturados. Ao fim iremos descrever também como foi a união desses dados estruturados e não-estruturados. O tratamento desses dados e as tecnologias utilizadas serão

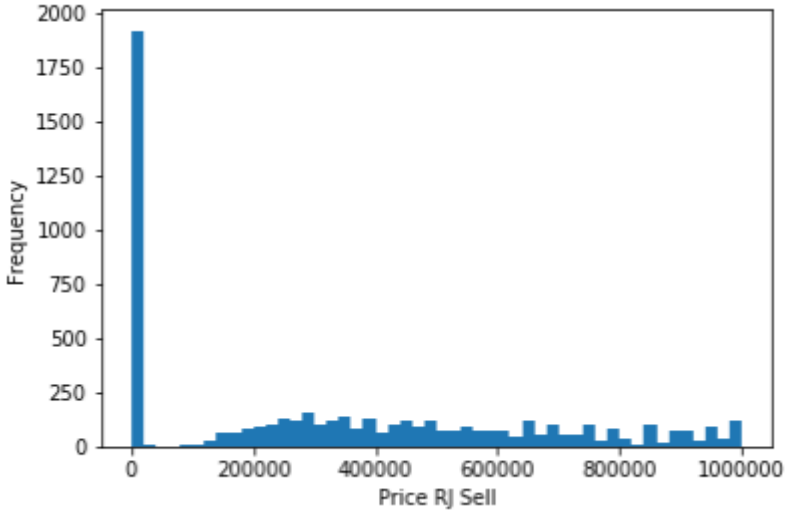
descritas nas próximas subseções. Todas a implementação do regressor foi feita em Python e suas bibliotecas adicionais.

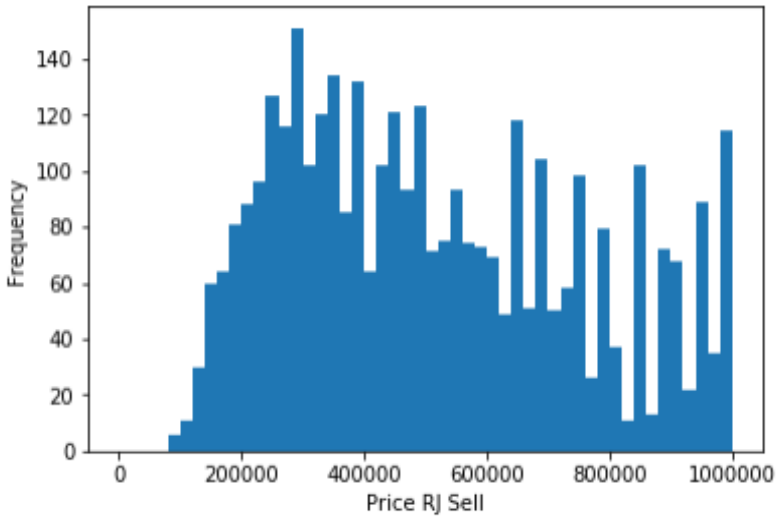
PRÉ-PROCESSAMENTO

Os dados estruturados em formato .csv foram carregados em memória utilizando a biblioteca de python Pandas. [r1618] Essa biblioteca transforma arquivos csv em Data Frames. Esses Data Frames funcionam como tabelas, onde os dados podem ser acessados através das colunas rotuladas anteriormente. Após a construção do Data Frame, foi feita uma análise dos dados presentes e foi visto que para o regressor colunas como a de "url"e "timestamp"não eram interessantes para o algoritmo. Portanto, essas colunas foram removidas. Além disso algumas colunas do Data Frame possuiam strings que descreviam o atributo.

Essas colunas eram "operation"e "type"que tinha como únicas entradas de string, respectivamente, "sell"e "rent", descrevendo venda ou aluguel, e "house"e "apartment", descrevendo o tipo do imóvel como casa ou apartamento. Essas colunas foram categorizadas e agora ficaram representadas pelas valores 1 e 0 ao invés das strings. Foi feita também uma verificação para valores nulos no Data Frame e as colunas e linhas com valores nulos foram removidos.

Percebeu-se também que os imóveis (ou linhas) que possuíam valor de "operation"igual a "rent", ou seja imóveis que estavam sendo ofertados como aluguel, estavam com valores de preço muito baixos. Portanto, foram classificados como outliers e foram removidos do Data Frame. Isso pode ser observado no exemplo do RJ nas Figuras 3.2 e 3.3 a seguir:





Após o tratamento dos dados e remoção de entradas indesejadas, foi feita a separação dos dados em conjuntos de treinamento e teste, onde foi dividido uma porcentagem de 80 (oitenta) por cento para treinamento e 20 (vinte) para teste. Assim, os dados de treinamento e teste já tratados foram gravados em novos arquivos csv para serem utilizados pelo script de criação do modelo do regressor.

EXTRAÇÃO E SELEÇÃO DE ATRIBUTOS DE TEXTO

Os dados não-estruturados foram recebidos em formato HTML, então foi utilizado uma biblioteca de Python chamada Beautiful Soup [r1718] para extrair todo o texto contido nas páginas html e assim grava-los em novos arquivos txt. Antes do processo de extração e criação dos arquivos txt, os arquivos HTML

foram ordenados para que os novos arquivos txt tivessem o mesmo index do imóvel indicado nos Data Frames já criados.

Esse processo facilitaria a integração dos dados estruturados e não-estruturados no futuro. Além disso, os arquivos txt foram separados em duas pastas diferentes que diferenciavam treinamento e teste, também de acordo com os data frames. Após a criação dos arquivos txt, foi utilizada uma pacote da bibliteoca Sci-Kit Learn [r1818], também utilizadas na criação de modelos, chamado TfidfVectorizer [r1918], para o carregamento dos arquivos txt em dois vetores. Uma para o conjunto de treinamento e outro para o conjunto de teste.

Esses vetores são utilizados para realizar o processo de seleção de atributos, já citado no capítulo 2, onde é delimitado uma quantidade máxima de features e também um arquivo que contenha stopwords, que são palavras de uso frequente que devem ser ignoradas pelo vetor na hora do calculo de relevância das palavras. A quantidade de atributos escolhida inicialmente para os vetores foi de 18000 (dezoito mil) atributos, porém por conta de alguns problemas, que serão discutidos em outra seção mais a frente, esse valor foi reduzido para 1000 (mil) atributos.

IMPLEMENTAÇÃO DOS MODELOS

Os modelos de regressão foram criados com a utilização da biblioteca Sci-kit Learn [r1818]. Essa biblioteca fornece vários regressores com algoritmos de aprendizado de máquina diferentes. Os algoritmos escolhidos foram: Random Forest, SVR e XGBoost. Inicialmente foram construídos dois modelos, um que recebia e utilizava os dados estruturados e outro que utilizava os não estruturados. Após o pré-processamento dos dados, rodar os regressores sobre esses dados utilizando o sci-kit learn foi bem simples. Os modelo foram construídos a partir do conjunto de treinamento e após isso foram testados sobre o conjunto de testes. O teste foi medido a partir das métricas citadas no capítulo 2. Os resultados e comparações sobre esses resultados serão analizados no capítulo 4. Após as duas versões iniciais, foi construído um novo modelo que unia os conjuntos de entrada de dados estruturados e não-estruturados. Iremos nos aprofundar nessa construção nas seções seguintes.

Primeiramente, foi implementado um regressor utilizando o algoritmo Random Forest, já citado no capítulo 2. O pacote utilizado foi o ensemble do Sci-kit Learn [r1818]. Os parâmetros do regressor foram definidos a partir do SMAC. O SMAC foi utilizado para otimizar o regressor ajustando os valores dos parâmetros para um melhor resultado do algoritmo. O SMAC foi somente usado para o Random Forest, pois tinha uma implementação mais simples e foi relevante para um levantamento inicial. Porém, aumentou muito o tempo de execução da criação do modelo. Isso resultou na opção de não utilizá-lo para os demais algoritmos de classificação.

Além do algoritmo de Random Forest, também foram utilizados regressores com os algoritmos SVR e XGBoost. Os parâmetros utilizados para o SVR foram os valores padrão definidos pelo Sci-kit Learn [r1818]. Foi utilizado para a implementação desse regressor o pacote SVM do Sci-kit learn. Já para o XGBoost, foi utilizado o pacote de Python xgboost e os parâmetros utilizados foram o padrão definido pelo pacote.

UNIÃO DOS CONJUNTOS DE DADOS

Nesta seção iremos discutir como foi implementado o regressor que utilizava os dois conjuntos de dados, estruturado e não-estruturado. O regressor teve uma implementação bem parecida com o dos dados estruturados. O Data Frame em pandas foi mantido, assim como os TfidfVectorizer. Porém, o vetor foi convertido em um novo Data Frame onde cada coluna representava uma feature do texto. Nas linhas desse Data Frame estavam os valores de TFIDF de cada uma das features e quando não houvesse valor para aquela feature em um certo documento o valor NaN era atribuído.

Após a construção desse novo Data Frame a partir do vetor foi feita a união dos dois Data Frames. Assim temos um novo Data Frame que possui os atributos estruturados e as features do texto como colunas. Após a união os valores NaN foram substituídos por 0.0, para assim não terem relevancia no calculo do regressor, mas serem contabilizados como dados.

Como dito na seção anterior, a limitação de features foi essencial para que o desempenho do algoritmo do regressor da união desses dados fossem

bom. Inicialmente foram definidas 10000 (dez mil) features para texto, porém o mesmo problema, que havia acontecido com POA anteriormente, aconteceu para os outros conjuntos de dados. Então, assim como para os dados não-estruturados, foi definido um limite de 1000 (mil) features para texto. O algoritmo executou bem para 1000 features, porém ainda teve um tempo de execução muito alto. Durou alguns dias para ser executado, mesmo para o conjunto do RJ que tinha uma quantidade de entradas bem menor (devido a remoção de imóveis para alugar).

DIFICULDADES E PROBLEMAS ENCONTRADOS

Nesta seção iremos detalhar alguns dos problemas encontrados durante a implementação do regressor, muitos dos problemas foram relacionados a estouro de memória e tempo de execução dos algoritmos. As próximas subseções irão detalhar também as soluções para estes problemas e o impacto deles na implementação final do algoritmo. Inicialmente, durante a implementação do regressor utilizando dados textuais, não foi configurado um limite para a quantidade de features (ou atributos).

Além disso, o vetor escolhido para definir a relevancia foi o CountVectorizer, que define a relevancia somente pela quantidade de repetições no documento. Isso impactou em uma quantidade de features muito grande, onde o maior número, que foi o de POA, tinha mais de 45000 (quarenta e cinco mil) features. Essa quantidade grande de features teve um impacto no desempenho do algoritmo. Quando executado demorou mais de 1 dia para terminar, e no caso de POA foi interrompido por um erro de estouro de memória. Então, foi decidido limitar a quantidade de features para 1800 (dezoito mil). Além disso, também foi preferível mudar o vetor sendo utilizado. Ou invés de utilizar o CountVectorizer, foi escolhido o TfidfVectorizer, que como dito anteriormente, conseguia definir melhor a relevância das palavras.

Após essas duas mudanças, os algoritmos ficaram muito mais rápidos, porém como a quantidade de dados textuais no conjunto de POA era maior do que os de SP e RJ, ainda assim não foi possível finalizar a execução. O erro de estouro de memória ainda persistiu. Então, foi decidido deixar esse conjunto de

dados de lado e trabalhar somente com os outros dois menores. Por questões de limitações da máquina, já que a quantidade de features continuava muito alta para o conjunto de POA. Porém, quando implementada a união dos conjuntos de dados, que irá ser detalhada na próxima seção, problemas semelhantes surgiram. Portanto, foi decidido limitar mais ainda o número de features para somente 1000 (mil). Essa mudança foi essencial para a execução do conjunto de dados de POA e não impactou tanto nos resultados que serão analisados no capítulo 4.

CAPÍTULO 4

ANÁLISE DOS RESULTADOS

Neste capítulo iremos analisar a configuração do ambiente e como os conjuntos de treinamento e teste de cada cidade foram divididos e também os resultados gerados por cada um dos regressores implementados.

Configuração do ambiente

Como já discutido no capítulo 3, os conjuntos de dados de cada cidade foram divididos em treinamento e teste para que fossem implementados os algoritmos de regressão. A divisão treinamento/teste foi dada por uma porcetagem de 80 por cento do conjunto de dados para treinamento e 20 por cento para teste. A divisão dos dados para todas as cidades estão descritos pela tabela 4.1.

Conjunto	RJ	SP	POA
Treinamento	4199	5875	7595
Teste	1009	1429	1894

Os regressores utilizados para predizer o preço dos imóveis foram: Random Forest, SVR e XGBoost. Para os dados não-estruturados foram utilizados vetores TFIDF, com 1000 atributos para a regressão. Quando foi feita a união das duas entradas, um novo data frame foi gerado sendo a união do data frame dos dados estruturados, já pré-processados, e de um data frame gerado a partir do vetor de 1000 atributos. Gerando assim um data frame com as colunas

originais já citadas e mais 1000 novas colunas para cada atributo extraído do pré-processamento textual.

Resultados

Nesta seção iremos mostrar e interpretar os resultados de cada um dos classificadores criados. Nas circunstâncias definidas na configuração do ambiente, e para cada um dos tipos de dados e sua combinação. Sendo possível assim uma comparação de cada um dos regressores a partir das métricas utilizadas, já citadas no capítulo 2.

Dados Estruturados

Os resultados do regressor Random Forest de dados estruturados estão demonstrados pela tabela 4.2.

Métrica	RJ	SP	POA
MSE	2.1501×10^{11}	2.7416×10^{11}	9.4513×10^{10}
MAPE	24.3408	30.5298	33.6817
MLE	0.2570	0.3216	0.3395
R2	0.6804	0.4680	0.3766

Podemos ver os resultados do regressor SVR de dados estruturados através da tabela 4.3.

Métrica	RJ	SP	POA
MSE	9.8832×10^{11}	8.6975×10^{11}	2.6728×10^{11}
MAPE	89.2273	79.3547	72.4214
MLE	0.6807	0.5880	0.5621
R2	-7.2925	0.0	-1.9722

A tabela 4.4 descreve os resultados do regressor XGBoost para os dados estruturados.

Métrica	RJ	SP	POA
MSE	2.1458×10^{11}	1.7106×10^{11}	5.8387×10^{10}
MAPE	25.2233	22.7950	25.5003
MLE	0.2436	0.2367	0.2570
R2	0.6788	0.6984	0.6834

A partir dos dados descritos nas tabelas, podemos observar que o regressor XGBoost obteve os melhores resultados para os conjuntos de dados de todas as cidades. Tendo valores de erro menores.

DADOS NÃO-ESTRUTURADOS

Os resultados dos dados não-estruturados para o regressor Random Forest pode ser visualizado a partir da tabela 4.5. O regressor SVR para dados não-estruturados apresentou os resultados descritos pela tabela 4.6. Os dados não-estruturados apresentaram os resultados descritos pela tabela 4.7 para o regressor XGBoost.

Métrica	RJ	SP	POA
MSE	8.9703×10^{11}	7.9540×10^{11}	2.4165×10^{11}
MAPE	66.2063	62.2558	57.9385
MLE	0.7447	0.6664	0.5910
R2	-63.1374	-107.1170	-107.1295

Métrica	RJ	SP	POA
MSE	9.8522×10^{11}	8.7004×10^{11}	2.6762×10^{11}
MAPE	88.5768	79.4119	72.6002
MLE	0.6807	0.5880	0.5622
R2	-1327337390969308.8	-3505266873730457.0	-946071832225996.8

Métrica	RJ	SP	POA
MSE	9.2185×10^{11}	8.1141×10^{11}	2.4528×10^{11}
MAPE	66.7736	62.8796	58.0812
MLE	0.7488	0.6668	0.5943
R2	-32.5899	-44.3245	-42.1948

A partir das tabelas apresentadas, podemos inferir que os resultados dos dados de texto foram bem piores do que os dados estruturados. Tendo o

conjunto com mais instâncias, POA, como o melhor dentre os outros. Com variações de conjunto para conjunto em cada um dos algoritmos.

Combinação dos Dados

Apesar dos resultados terem sido piores para os dados textuais, quisemos verificar como a combinação dos dados textuais iriam interferir nos resultados dos dados estruturados, e se seria possível uma melhora nos resultados. A tabela 4.8 descreve os resultados do regressor Random Forest para a combinação dos dados estruturados e não-estruturados.

Os resultados do regressor SVR para a combinação dos dados estruturados e não-estruturados podem ser visualizados na tabela 4.9 A combinação dos dados estruturados e não-estruturados apresentaram os resultados descritos pela tabela 4.10 para o regressor XGBoost. Podemos observar nas tabelas apresentadas que obtivemos uma melhora não muito significativa em algumas métricas para os regressores apresentados nas Tabelas 4.8, 4.9 e 4.10.

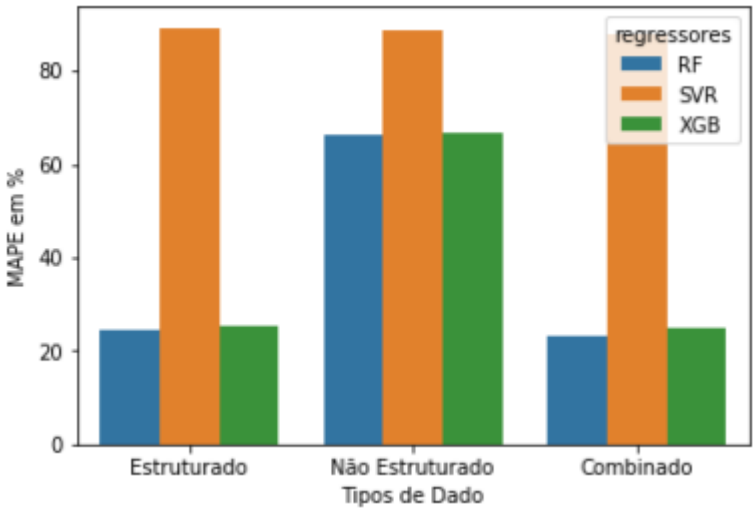
Iremos analisar mais afundo como foram essas mudanças apresentando os gráficos contidos nas Figuras 4.1, 4.2 e 4.3, para comparar os resultados obtidos com cada um dos tipos de entrada de dados.

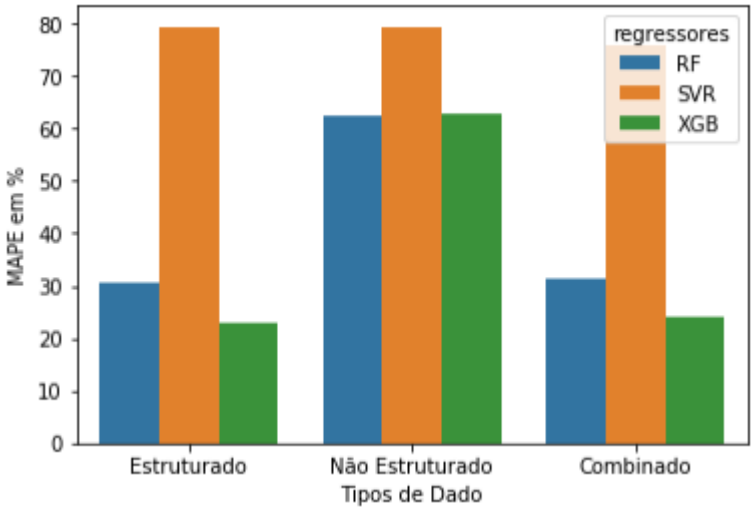
Métrica	RJ	SP	POA
MSE	1.7831×10^{11}	2.8138×10^{11}	6.4852×10^{10}
MAPE	23.2393	31.4526	26.4236
MLE	0.2444	0.3328	0.2661
R2	0.7446	0.4133	0.6301

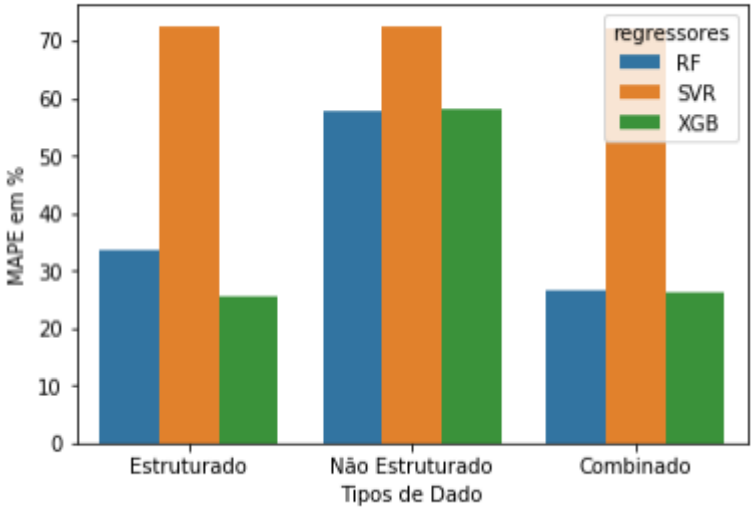
Métrica	RJ	SP	POA
MSE	9.0878×10^{11}	7.7300×10^{11}	2.6268×10^{11}
MAPE	87.98	75.8377	71.9970
MLE	0.6871	0.5746	0.5615
R2	-4.6710	-1.5121	-7928168721069216.0

Métrica	RJ	SP	POA
MSE	1.9145×10^{11}	1.7747×10^{11}	6.2604×10^{10}
MAPE	24.7457	23.8949	26.1248
MLE	0.2626	0.2534	0.2653
R2	0.7026	0.6375	0.6476

Podemos ver a partir dos gráficos, que os resultados dos dados estruturados e combinados tem valores semelhantes ou próximos. O que nos indica que de o impacto dos dados nãoestruturados na regressão foi bastante baixo. Podemos ver também a discrepância dos valores para somente os dados não-estruturados em relação aos outros tipos. Assim como, também podemos ver que o regressor SVR teve desempenho bem abaixo dos demais.







CAPÍTULO 5

Conclusão

Neste trabalho foram apresentados vários regressores com o intuito de prever o preço de imóveis, de acordo com os conjuntos de dados de 3 grandes cidades brasileiras, como também de comparar os resultados dos dados estruturados e não-estruturados nessa predição e qual o impacto dos dados não-estruturados quando a combinação deles é utilizada. Além disso, foi apresentado o pré-processamento tanto dos dados estruturados, quanto dos dados não-estruturados. Realizando processos como processamento de texto e análise de data frames.

Os algoritmos de regressão utilizados tiveram resultados bons para os dados não estruturados, tendo como o melhor deles o XGBoost ficando na frente de Random Forest e SVR, nesta ordem. Para os dados não-estruturados, notamos que os conjuntos que tinham mais instâncias tiveram resultados melhores para todos os algoritmos. Como o conjunto com mais instancias era POA, tivemos o melhor resultado desse conjunto no algoritmo de Random Forest. Para a combinação dos dados estruturados tivemos resultados que não foram muito expressivos em relação aos que tinham sido obtidos somente com os dados estruturados. Isso indica que o impacto dos dados não-estruturados sobre os estruturados na definição do preço dos imóveis é mínima. Tivemos

como os melhores regressores o XGBoost e o Random Forest, que tiveram resultados bem próximos. Já o SVR teve um desempenho muito ruim, sendo até distoante dos demais regressores.

Como a quantidade de features geradas pelos vetores de texto foi muito grande, limitou-se as features do vetor dessa forma obteve-se menos informação sobre os dados textuais, o que pode ter prejudicado o desempenho dos regressores que utilizaram dados textuais. Porém, a limitação de features foi necessária para que o algoritmo conseguisse rodar em tempo hábil. Tanto quando utilizou somente os dados textuais, e principalmente quando utilizou a combinação dos dois. O problema da quantidade de features se agrava para a combinação pois a entrada se torna um data frame com uma quantidade de colunas igual a de features mais 8, que eram as que já existiam inicialmente. Isso torna o conjunto de dados com uma quantidade muito grande de colunas, o que resulta em tempo de execução maior.

Para estudos futuros, seria possível avaliar quais são essas features que estão sendo escolhidas e se a relevancia aplicada a ela está sendo aplicada corretamente pelo algoritmo. Além disso, também seria interessante conduzir um estudo que avaliasse a partir consumidores que atributos presentes, tanto nos dados estruturados quanto nos textos de descrições, avaliados nos dados não-estruturados, seriam relevantes para eles na valorização de um imóvel. Isso traria um novo viés de rankeamento que poderia definir pesos mais precisos para a relevancia desses atributos, o que, por fim, também aumentaria a precisão do algoritmo em relação a previsão de preço.

ESTUDO DE CASO II

Predição de Alunos com Risco de Evasão: estudo de caso usando mineração de dados

Introdução

As instituições brasileiras têm apresentado altos índices de evasão em seus cursos superiores. Esse fato tem causado preocupação aos gestores acadêmicos, que buscam melhorar os indicadores de retenção e conclusão de curso. Nesse contexto, surge a importância de identificar antecipadamente quais estudantes não terão êxito na conclusão do curso, pois essa informação pode auxiliar na tomada de decisões para que essa previsão possa ser modificada [Oliveira et al. 2017].

Para isso, podem ser utilizadas técnicas de Mineração de Dados Educacionais (Educational Data Mining - EDM). Essas técnicas são capazes de identificar o perfil desses estudantes a partir de dados de outros estudantes que não foram bem sucedidos na universidade [Baker et al. 2011].

A Universidade Federal do Rio Grande (FURG) é uma das instituições brasileiras que tem enfrentado o desafio de reduzir o elevado número de evasão, pois muitos alunos abandonam seus cursos durante a trajetória acadêmica. Por isso, existe a necessidade do uso de técnicas de EDM para auxiliar na detecção antecipada da evasão escolar, permitindo que suas possíveis causas possam ser analisadas e tratadas prematuramente.

Nesse sentido, este trabalho tem como objetivo identificar o subconjunto dos alunos de graduação da FURG que apresentam risco de evasão. Para isso, foram analisadas diferentes informações dos alunos que evadiram ou concluíram seus cursos de graduação no período de 2012 a 2017, visando gerar um modelo capaz de classificar corretamente novas instâncias de alunos como sendo ou não um possível evasor. Espera-se que o referido modelo auxilie os gestores da FURG no desenvolvimento de ações para reduzir o abandono de curso. A previsão da evasão escolar é um tema já estudado por diversos pesquisadores [Manhães et al. 2011, Brito et al. 2015, Digiampietri et al. 2016].

No entanto, este trabalho apresenta uma abordagem diferente destes, pois busca compreender a evasão analisando dados de alunos que já cursaram no mínimo um ano do curso de graduação, a partir de dados demográficos e de desempenho acadêmico. O restante do trabalho está organizado da seguinte

forma: na Seção 2 é apresentada a metodologia empregada no estudo; na Seção 3 estão descritos os resultados obtidos; e a Seção 4 apresenta as conclusões e os trabalhos futuros.

METODOLOGIA

A base de dados utilizada neste trabalho foi coletada do sistema acadêmico da FURG, contendo informações sobre alunos de 12 cursos de graduação de diferentes áreas do conhecimento. É importante ressaltar que os alunos tiveram a sua identificação preservada, sendo a matrícula o atributo usado para relacionar os dados.

Os dados foram extraídos em diferentes arquivos no formato XLS, e importados para um banco de dados MySQL. Para compor o dataset final somente foram considerados os alunos que ingressaram pelo ENEM/SISU, que evadiram ou concluíram o curso, e aqueles que já tinham cursado no mínimo um ano do curso.

Esse último critério foi utilizado devido ao atributo índice de rendimento acadêmico, que na maioria dos casos se obtém um valor significativo após o primeiro ano no curso de graduação. Como resultado dessa filtragem, obteve-se 916 registros, onde 720 pertencem a classe evadido e 196 a classe concluinte.

Além disso, os atributos numéricos foram discretizados e as faixas de valores foram usadas para criar as seguintes categorias:

• Faixa de idade do aluno no momento da evasão ou conclusão do curso: ID1 (16- 20); ID2 (21-25); ID3 (26-30); ID4 (mais de 30 anos).

• Média geral do ENEM: ME1 (400-499); ME2 (500-599); ME3 (600-699); ME4 (maior ou igual a 700).

• Intervalo de tempo entre a conclusão do ensino médio e o ingresso no curso de graduação: II1 (menor que 2 anos); II2 (2-4 anos); II3 (5-9 anos); II4 (maior ou igual a 10 anos).

• Último coeficiente de rendimento: CO1 (menor que 5,0); CO2 (5,0-6,9); CO3 (7,0-8,9); CO4 (maior ou igual a 9,0).

Já para o atributo representante do curso foram criadas 4 categorias de acordo com a área de conhecimento do mesmo, sendo elas: ciências da saúde - AC1 (Educação Física, Enfermagem), ciências exatas - AC2 (Engenharia Civil, Engenharia de Computação, Engenharia Química), ciências humanas - AC3 (História Licenciatura, Pedagogia, Psicologia, Geografia) e ciências sociais aplicadas - AC4 (Administração, Ciências Contábeis, Direito). Os demais atributos referem-se ao gênero (M ou F), tipo da escola de origem (Pública ou Privada), se o aluno foi ou não bolsista em algum momento da graduação, estado de origem (Rio Grande do Sul ou outro estado). Por fim, a situação final do aluno no curso (evadido ou concluinte) foi usada como classe alvo.

Para o processo de mineração de dados utilizou-se a ferramenta de código aberto Weka [Hall et al. 2009]. Após a etapa de pré-processamento, a tarefa de classificação foi realizada utilizando o algoritmo J48 [Quinlan 1993] para processar o dataset e gerar uma árvore de decisão. A tarefa de classificação utiliza algoritmos de aprendizagem de máquina para rotular, automaticamente, novas instâncias de uma base de dados com uma determinada classe, aplicando um modelo previamente aprendido baseado no valor dos atributos das instâncias de treinamento [Tan et al. 2005]. Neste trabalho, foi utilizado o algoritmo J48 por apresentar resultados intuitivos e, consequentemente, permitir melhor entendimento por parte dos gestores em relação aos valores dos atributos e a situação final de cada aluno no curso (concluinte ou evadido).

O referido algoritmo foi configurado para usar como critério de poda somente os nós que possuíam a partir de 10 instâncias (parâmetro minNumObj), visando evitar que a árvore ficasse muito grande. Além disso, esse parâmetro proporcionou uma melhor acurácia na classificação. Para os demais parâmetros, foi mantida a configuração padrão do Weka. O modelo de classificação gerado foi testado utilizando 10 partições na validação cruzada.

Resultados Obtidos

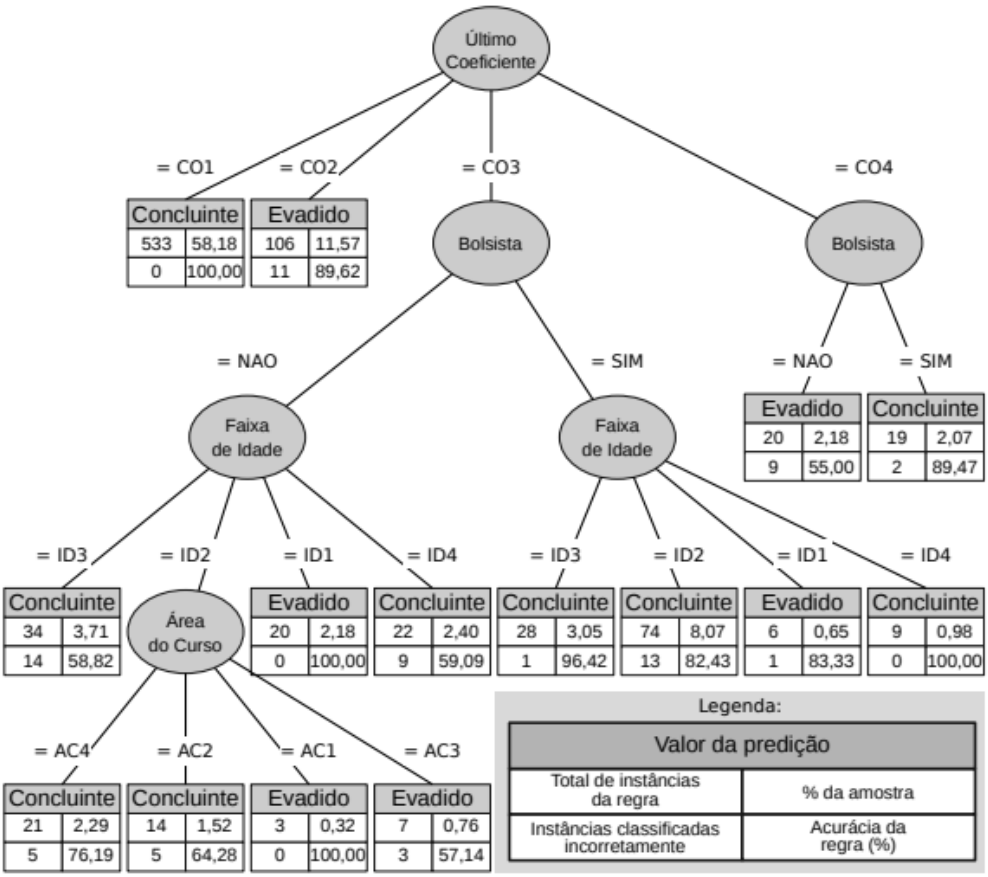
A Figura 1 mostra as regras geradas pelo algoritmo J48. Nela, é possível observar que o valor do atributo coeficiente de rendimento é significativo para

gerar o modelo de classificação, pois os alunos com coeficiente menor que 7,0 (categorias CO1 e CO2) são classificados na situação “Evadido”, representando mais de 60% da amostra, tendo taxa de acerto de 100%. Ou seja, alunos com coeficiente de rendimento baixo tendem a abandonar o curso, independente de outras características.

Já os alunos com coeficiente maior ou igual a 9,0 e que foram bolsistas durante a graduação, independente do período de duração da bolsa, são classificados como “Concluinte”, e os não bolsistas como “Evadido”. Isso demonstra que, mesmo o aluno possuindo um bom coeficiente de rendimento, o envolvimento em atividades extraclasse é muito importante para a permanência do aluno na universidade.

De acordo com o modelo gerado, para os alunos com coeficiente de rendimento maior ou igual a 7,0 e menor que 9,0 (categoria CO3), independente de ter sido bolsista ou não, a sua faixa de idade influência diretamente na situação final no curso, pois os alunos bolsistas com faixa de idade acima de 20 anos (categorias ID2, ID3 e ID4) são classificados como “Concluinte”, mas com idade entre 15 e 20 anos (categoria ID1) tendem a evadir do curso. Em corroboração, os alunos que se enquadram na categoria CO3, não bolsistas, e com faixa de idade acima de 25 anos (categorias ID3 e ID4), são classificados como “Concluinte”. Já os alunos da categoria ID1 são classificados como “Evadido” com uma acurácia de 100%. Com isso, percebe-se que os alunos mais jovens, com coeficiente intermediário, estão mais propensos a evasão.

O atributo área do curso é analisado para aqueles alunos com coeficiente de rendimento da categoria CO3, não bolsistas e com idade maior que 20 e menor ou igual a 25 anos (categoria ID2). Nesse caso, são classificados como “Evadido” os alunos da área das ciências da saúde (AC1) ou das ciências humanas (AC3), e como “Concluinte” sendo da área das ciências exatas (AC2) ou ciências sociais aplicadas (AC4). Essa informação demonstra que a área de conhecimento do curso não está diretamente ligada a evasão, pois primeiramente são analisadas outras características dos alunos.



Para a amostra de dados e configuração usada nesse experimento, percebe-se que os valores da média do ENEM, o estado de origem do aluno, o tipo de escola de origem, o gênero, e o intervalo para ingresso na graduação não foram considerados relevantes para gerar o modelo de classificação. Ou seja, não são usados diretamente como indicadores para detectar um aluno como sendo ou não um possível evasor. No entanto, observa-se que a faixa de idade apresentou grande influência para essa predição, o que demonstra que a utilização de dados demográficos pode contribuir para a obtenção de modelos cada vez mais precisos. De acordo com os resultados experimentais, o algoritmo J48 apresentou acurácia de 90,7%, mostrando ser viável a identificação dos alunos que apresentam risco de evasão através do uso de técnicas de mineração de dados.

CONCLUSÃO E TRABALHOS FUTUROS

O elevado índice de evasão escolar é um sério desafio enfrentado pelas instituições de ensino superior. A identificação precoce de alunos com este perfil

permite o planejamento de ações com propósito de evitar o abandono do curso. Neste trabalho, foi utilizado o algoritmo J48 na identificação de alunos com risco de evasão com taxas de acerto de 90,7%. Desse modo, espera-se que esse modelo de classificação proporcione aos gestores educacionais da FURG indicadores que possam ser efetivamente utilizados para reduzir os índices de evasão. Como trabalhos futuros pretende-se tratar a distribuição desbalanceada entre as classes, e realizar experimentos usando os dados dos demais cursos de graduação da FURG. Dessa forma, poderão ser realizados estudos mais detalhados por curso, bem como estudos comparativos. Além disso, poderá ser verificado a eficiência na predição da evasão com base em outras informações acadêmicas dos alunos, como a frequência, o registro de empréstimos na biblioteca, e o desempenho nas disciplinas.

REFERÊNCIAS

Eric Medvet Alberto Bartoli, Andrea De Lorenzo and Fabiano Tarlao. Inference of regular expressions for text extraction from examples. IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING, 28(5), May 2016.

Indresh Bhattacharyya. Support vector regression or svr. Medium, June 2018.

Jason Brownlee. A gentle introduction to xgboost for applied machine learning. August 2016.

Sylvain Chartier Denis Cousineau. Outliers detection and treatment: a review. International Journal of Psychological Research, 3(1), 2010.

Pasi Tapanainen Gregory Grefenstette. What is a word, what is a sentence? problems of tokenization. Rank Xerox Research Centre Grenoble Laboratory, April 1994.

Trevor Hastie Robert Tibshirani Gareth James, Daniela Witten. An Introduction to Statistical Learning with Applications in R. 2013.

Suhit Gupta, Gail Kaiser, David Neistadt, and Peter Grimm. Dom-based content extraction of html documents. Proceedings of the 12th International Conference on World Wide Web, pages 207–214, 2003.

Roger Eeckels Kobus Herbst Jan Van den Broeck, Solveig Argeseanu Cunningham. Data cleaning: Detecting, diagnosing, and editing data abnormalities. September 2005.

S. B. Kotsiantis. Supervised machine learning: A review of classification techniques. Proceedings of the 2007 Conference on Emerging Artificial Intelligence Applications in Computer Engineering: Real Word AI Systems with Applications in eHealth, HCI, Information Retrieval and Pervasive Technologies, (22):3–24, 2007.

Hong-Bo Shi Li-Ping Jing, Hou-Kuan Huang. Improved feature selection approach tfidf in text mining. Proceedings. International Conference on Machine Learning and Cybernetics, November 2002.

Mercari. Price suggestion challenge: Can you automatically suggest product prices to online sellers? 2018. [Pyl99] Dorian Pyle. Data Preparation for Data Mining. 1999.

Liangliang Cao Jiebo Luo Quanzeng You, Ran Pang. Image-based appraisal of real estate properties. IEEE Transactions on Multimedia, 19(12):2751 – 2759, December 2017.

Python data analysis library. 2018. [r1718] Beautiful soup. 2018. [r1818] Sci-kit learn. 2018. [r1918] Tfidfvectorizer. 2018.

Susan Spiggle. Analysis and interpretation of qualitative data in consumer research. Journal of Consumer Research, 21(3):491–503, December 1994.